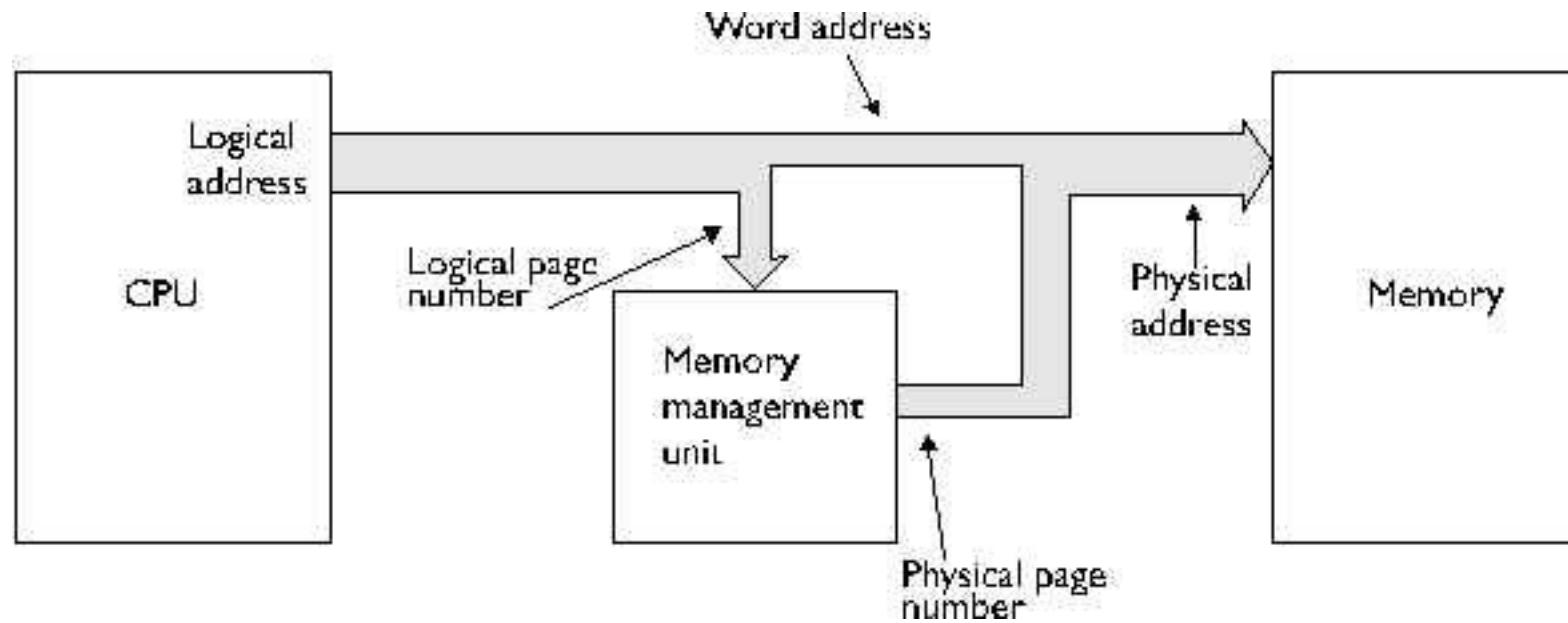


Administración de memoria



memoria

- Para que un proceso pueda ejecutarse debe estar ubicado en la memoria principal del ordenador.
- Una parte del sistema operativo se va a encargar de gestionar la memoria principal,
 - de forma que los procesos puedan residir en la memoria sin conflictos.
- Implica varias tareas,
 - registro de qué zonas están libres y qué zonas están ocupadas por qué procesos.
 - sistemas en los que no todos los procesos, o no todo el código y datos de un proceso, se ubican en la memoria principal.
 - carga de los programas de disco a memoria
 - el de la protección.

Organización de la memoria

Real	Real		Virtual	
Mono usuario	Multiprogramación		Multiprogramación	
	Particiones		Paginación pura	Segmentación pura
	Fijas	Variables	Combinación	
	Relocalización		Protección	

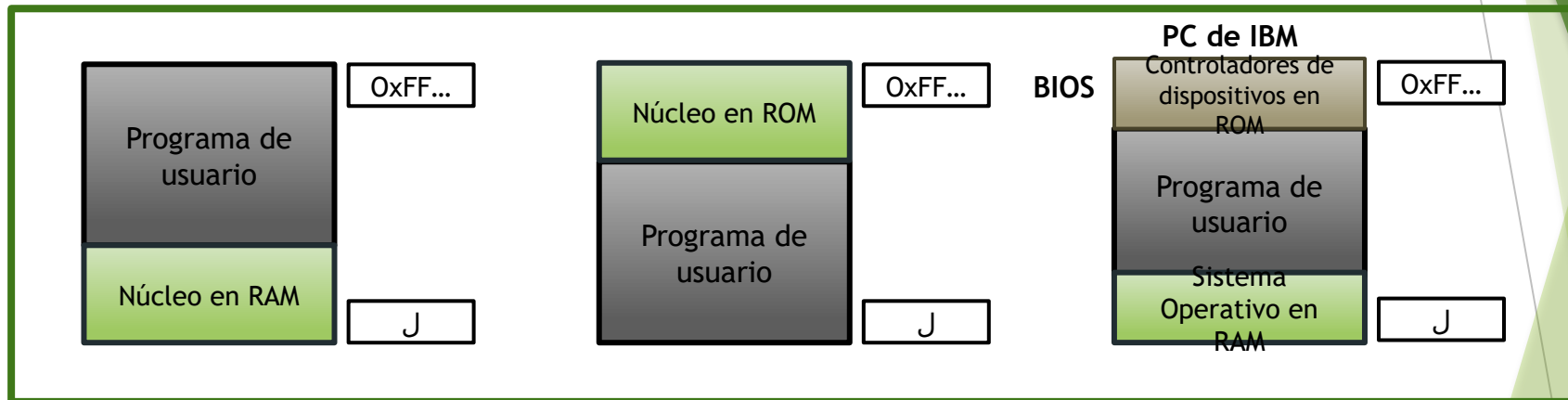
PAÑORAMA DEL MANEJO DE MEMORIA

MANEJO DE MEMORIA EN SISTEMAS MONOUSUARIO SIN INTERCAMBIO.

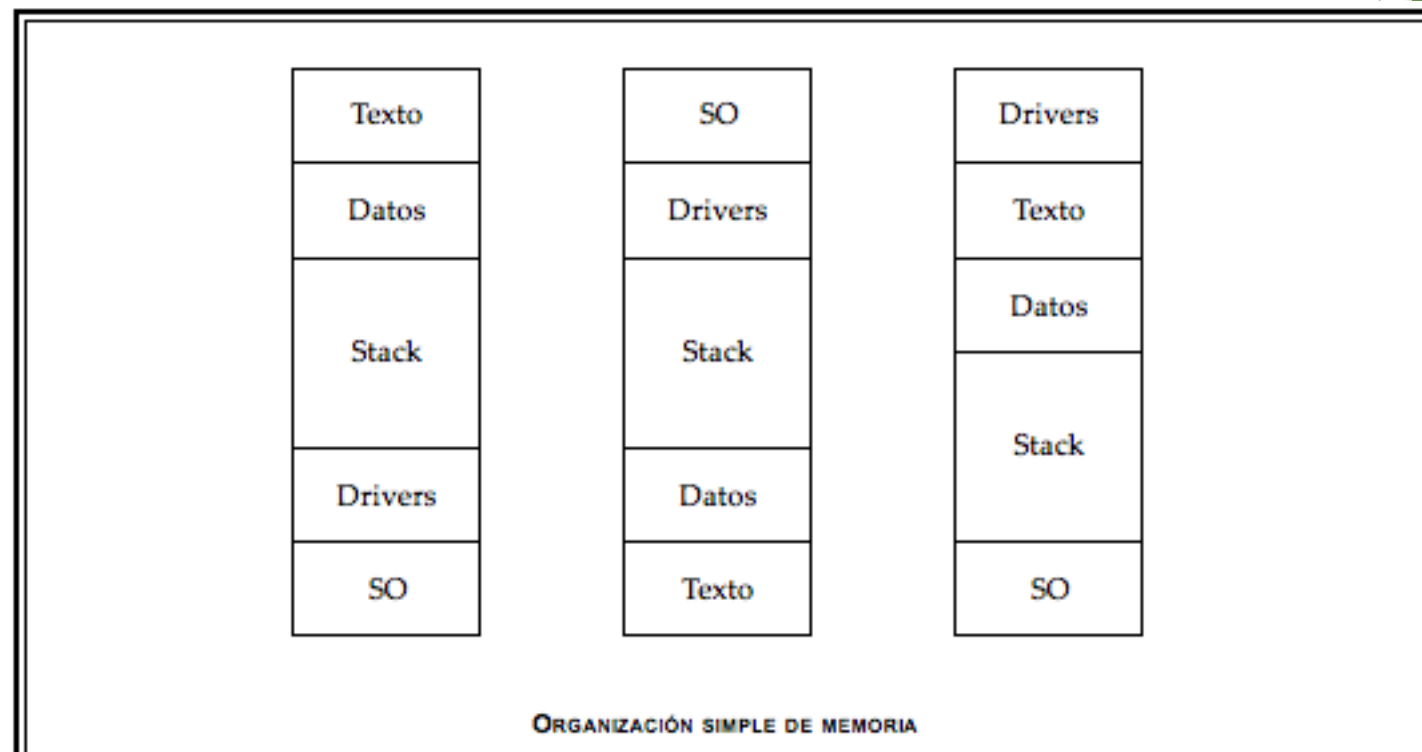
- el sistema operativo se ubica en las localidades superiores o inferiores de la memoria,
- seguido por algunos manejadores de dispositivos (drivers).
- Esto deja un espacio contiguo de memoria disponible que es utilizado por los programas del usuario,
- dejando generalmente la ubicación de la pila (stack) al último, con el objetivo de que ésta pueda crecer hasta el máximo posible.

Administración de la memoria en los sistemas monoprogramados

Tres formas de organización de la memoria RAM, con un sistema operativo y un proceso de usuario. Todo el programa es cargado a memoria para su ejecución



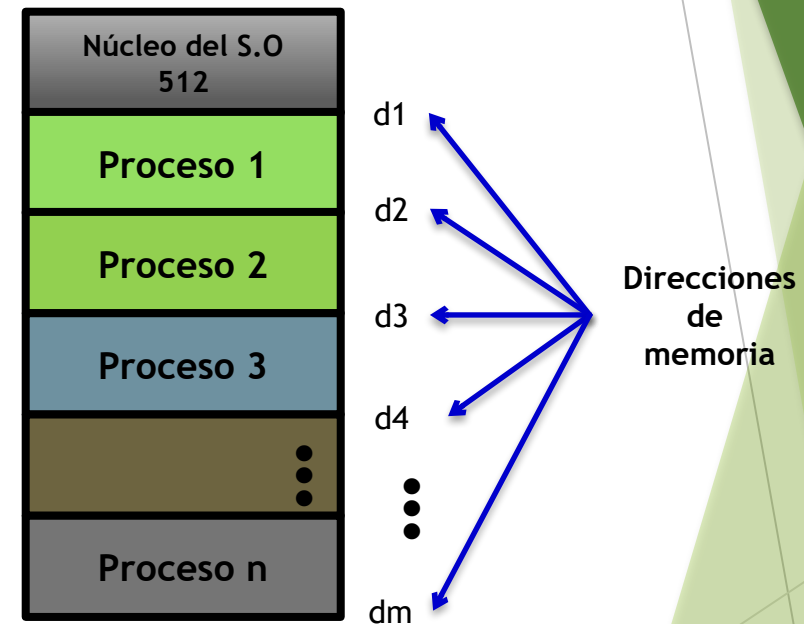
Organización simple



Administración de la memoria en los sistemas multiprogramados

Los problemas que aparecen en los sistemas multiprogramados y/o multiusuarios son:

1. Llevar un recuento de las zonas de memoria ocupadas por los procesos. ¿Qué proceso ocupa cual espacio?.
2. El momento de escribir un programa no se sabe en qué zona de memoria se ubicará, siendo posible que durante la vida de un proceso éste cambie varias veces de lugar.
3. La protección de las zonas de memoria ocupadas por los procesos, máxime en sistemas multiusuario donde los procesos pueden pertenecer a distintos usuarios.



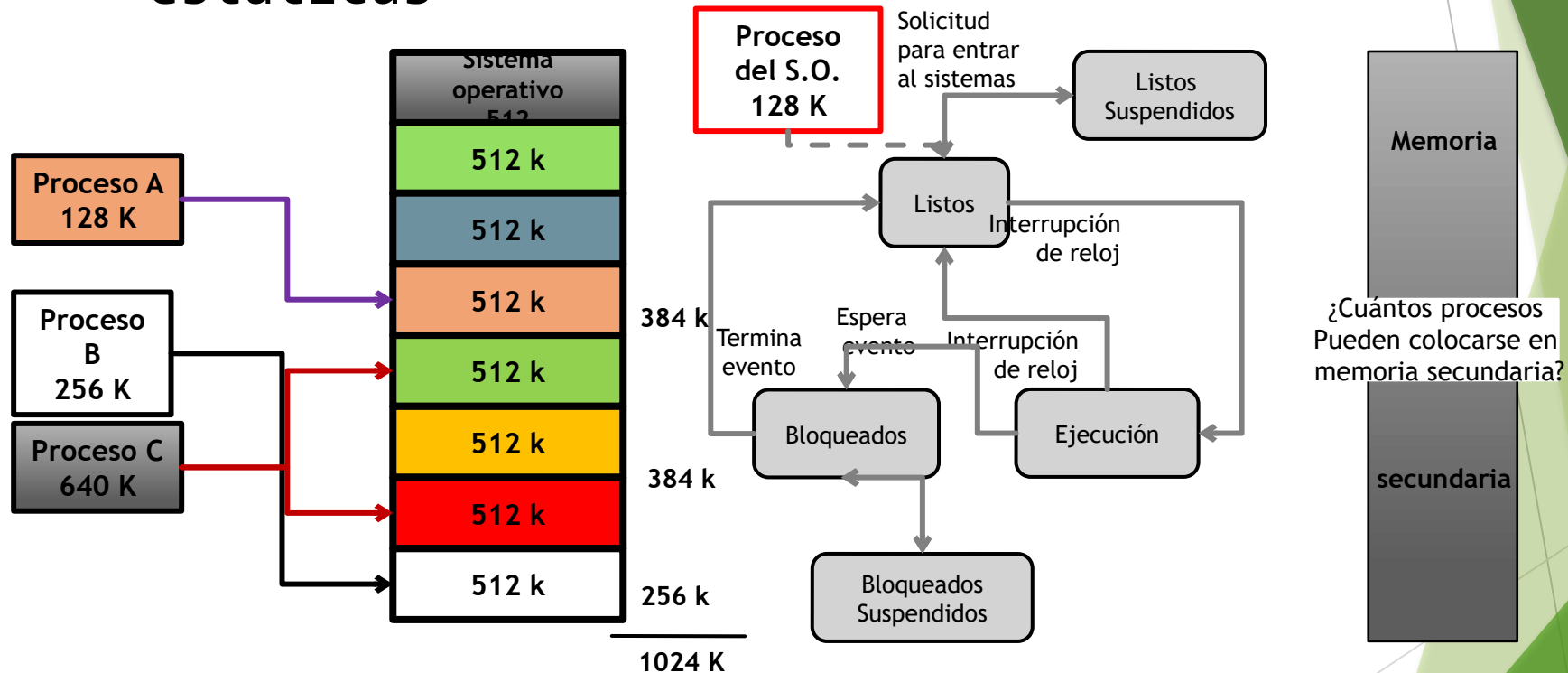
MULTIPROGRAMACIÓN EN MEMORIA REAL..

- Mas de un proceso al mismo tiempo
 - El problema de la relocalización.
 - La solución «estática» consiste en que todas las direcciones del programa se vuelvan a recalcular al momento en que el programa se carga a memoria, esto es, prácticamente se vuelve a recompilar el programa.
 - La solución «dinámica» consiste en tener un registro que guarde la dirección base de la sección que va a contener al programa. Cada vez que el programa haga una referencia a una dirección de memoria, se le suma el registro base para encontrar la dirección real.

Asignación de memoria contigua

Particiones estáticas

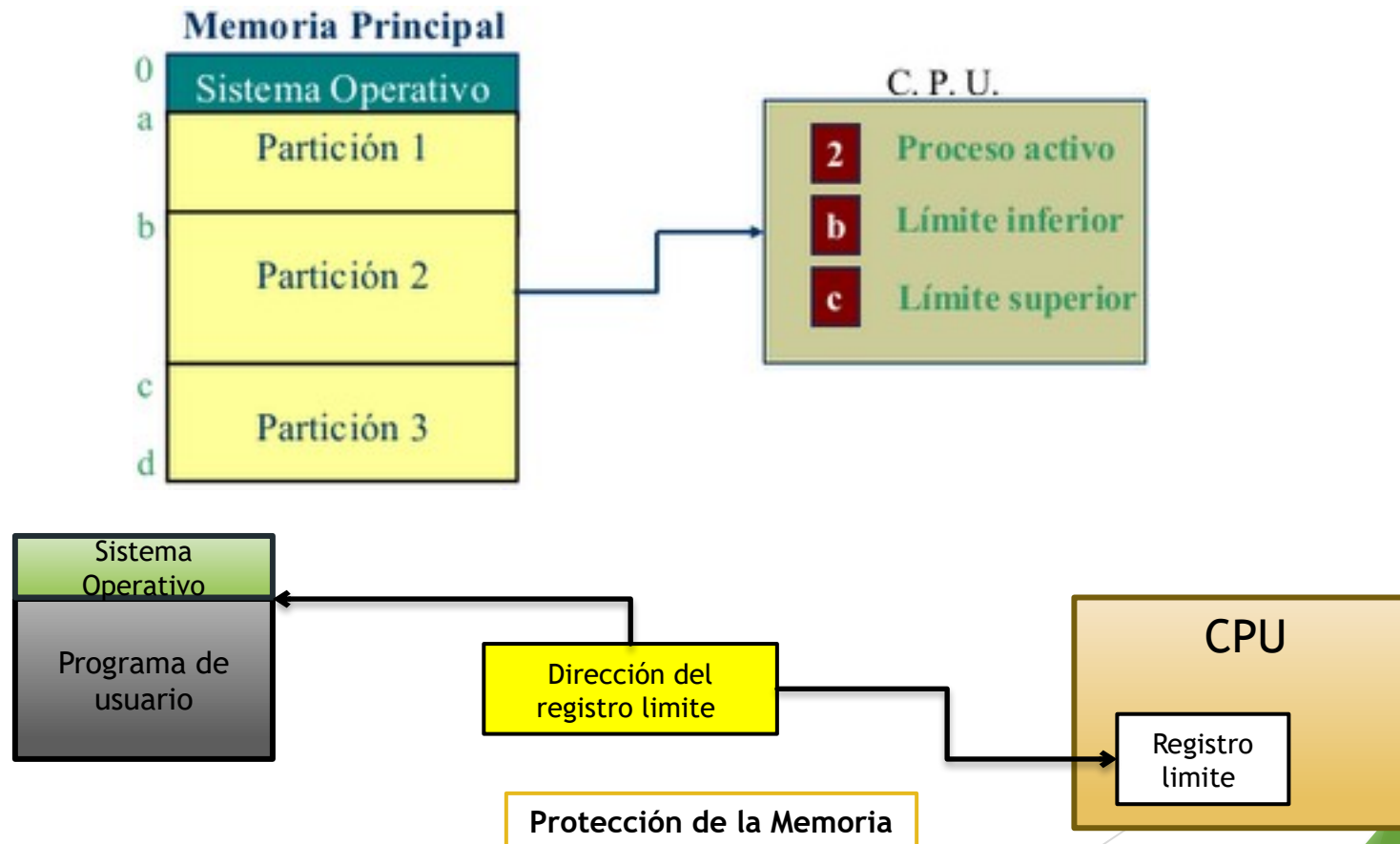
Los trabajos se traducían mediante compiladores y ensambladores absolutos, para ejecutarse en una partición específica



Una vez introducido un proceso en una partición, permanece en ella hasta su finalización aunque se encuentren libres otras particiones. Esto provoca una pérdida de eficiencia

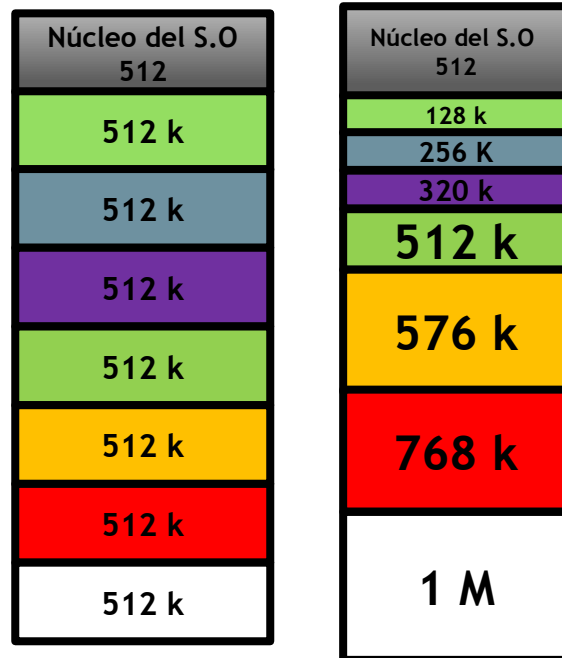
MULTIPROGRAMACIÓN EN MEMORIA REAL..

- Mas de un proceso al mismo tiempo
 - El problema de la protección.



Asignación de memoria contigua

Particiones estáticas



Organización de la memoria:

La memoria se divide en regiones con límites fijos, con dos alternativas:

1. particiones igual tamaño o
2. particiones de diferentes tamaños.

Asignación de procesos a la memoria:

Se pueden tener dos esquemas de asignación de memoria en particiones estáticas utilizando colas de procesos:

1. Colas de planificación para cada partición.
2. Una cola de planificación para la memoria (mayor eficiencia)

PARTICIONES FIJAS.

- particiones se crean cuando se enciende el equipo permanecen con los tamaños iniciales hasta que el equipo se reinicie.
 - Es una alternativa muy vieja, las particiones se realizaban por el operador analizando los tamaños estimados de los trabajos de todo el día.

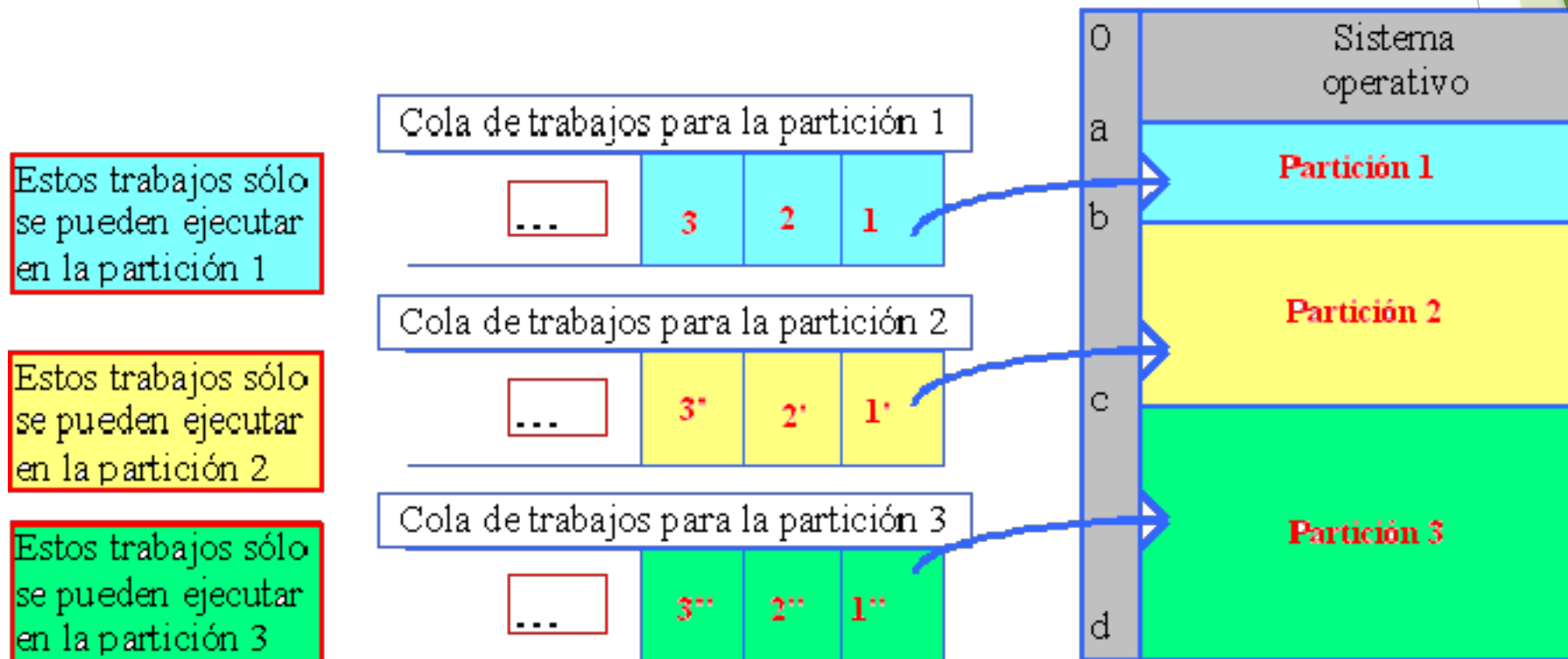
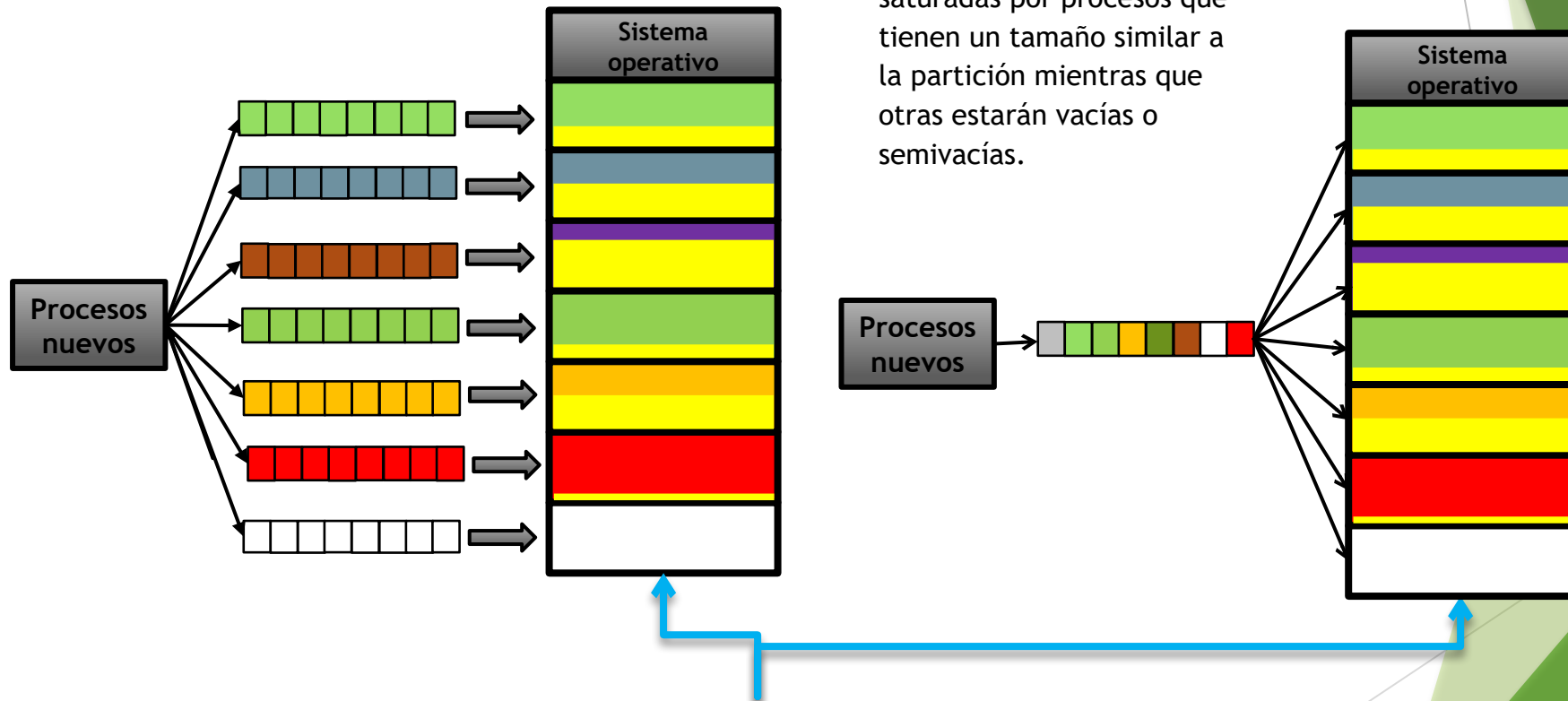


Figura 6.3. Multiprogramación con particiones estáticas, con traducción y carga absolutas

Asignación de memoria contigua

Particiones estáticas



Cuando un proceso se introduce en una partición, lo más probable es que su tamaño no sea el mismo (es decir, sea algo menor) que el de la partición. Esto origina un problema de desperdicio de memoria conocido como **fragmentación interna**.

PARTICIONES DINAMICAS

- particiones contiguas de tamaño variable.
 - una estructura de datos suficiente
 - dónde había huecos disponibles de RAM
 - dónde había particiones ocupadas por programas en ejecución.
 - Así, cuando un programa requería ser cargado a RAM,
 - el sistema analizaba los huecos para saber si había alguno de tamaño suficiente para el programa entrante,
 - si era así, le asignaba el espacio.
 - Si no, intentaba relocalizar los programas existentes .
 - Fragmentación externa

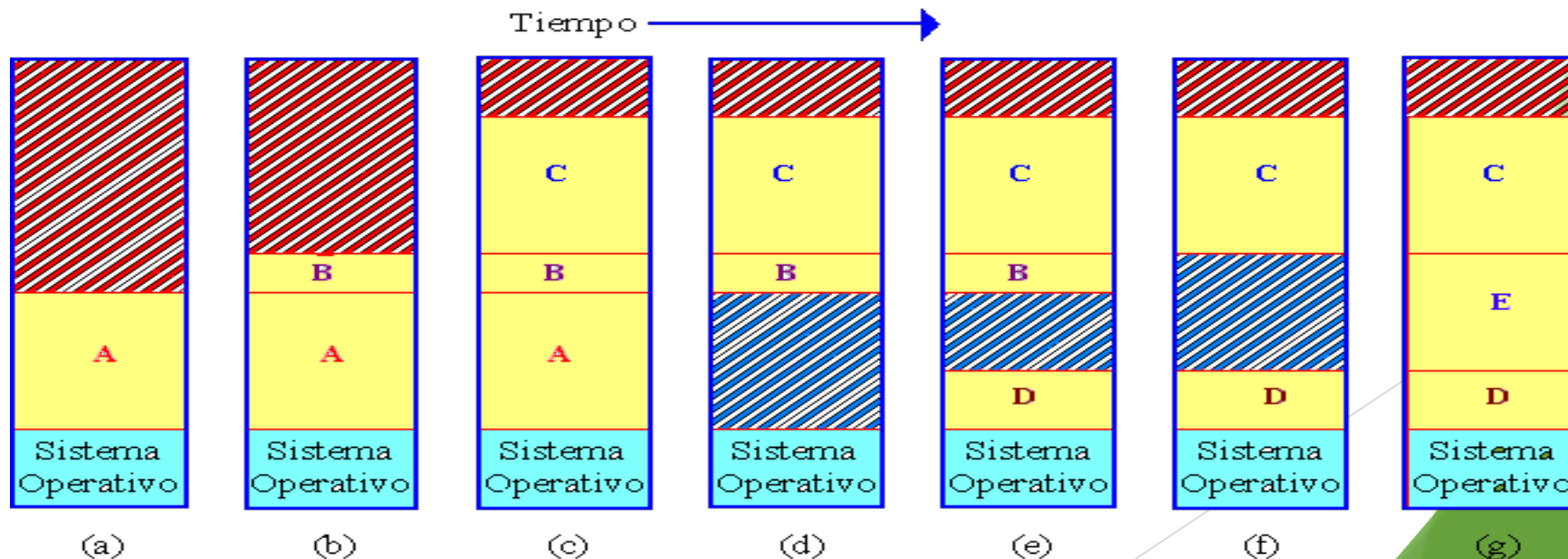
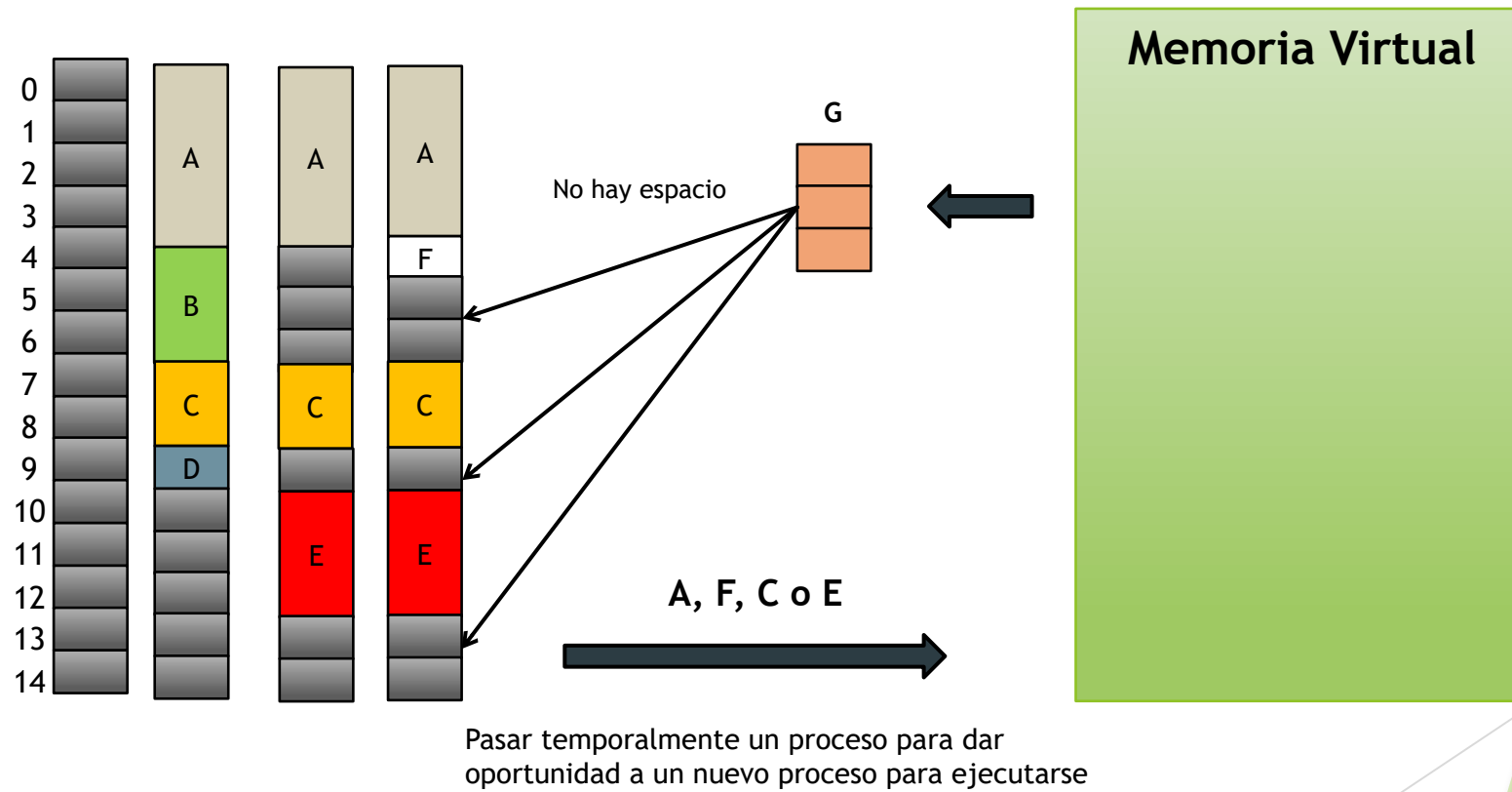


Figura 6.4 La asignación de memoria cambia cuando el proceso llega o sale de la memoria.

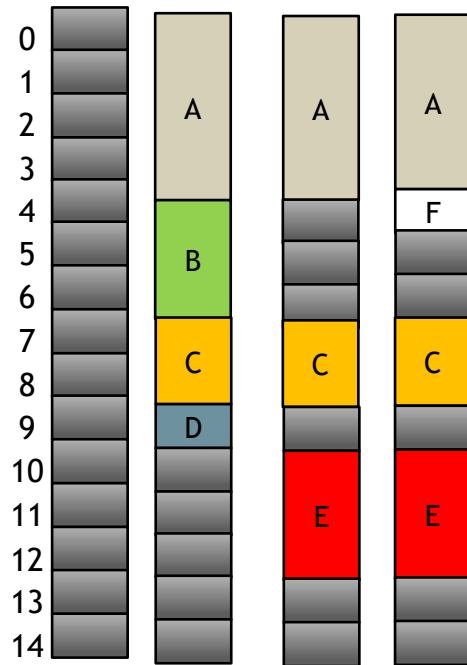
Asignación de memoria contigua

Particiones dinámicas



Asignación de memoria contigua

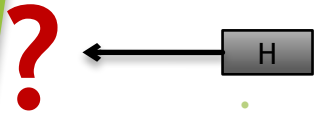
Particiones dinámicas



El continuo ingreso y salida de los procesos producen huecos en la memoria demasiado pequeños para que un proceso pueda colocarse. A este tipo de fragmentación se le conoce como **fragmentación externa**.

La recuperación de los huecos de memoria puede llevarse a cabo por medio de la técnica de compactación donde los espacios disponibles de memoria son colocados en la parte inferior y los procesos en la parte superior. Sin embargo, este proceso es demasiado lento y posiblemente no justifique el consumo de recursos, debido a que el sistema debe estar inactivo para poder realizar la compactación.

Problema de la colocación.



- En relación a los esquemas de particiones fijas y particiones variables surgen varios problemas:
 - ¿en base a qué criterio se elige el mejor tamaño de partición para un programa?
 - por ejemplo, si el sistema tiene dos huecos, uno de 18K y otro de 24K para un proceso que desea 15K, ¿cuál se le asigna?

- varios algoritmos para dar respuesta a la pregunta anterior

- Primer ajuste: se asigna el primer hueco que sea mayor al tamaño deseado.
- Mejor ajuste: se asigna el hueco cuyo tamaño exceda en la menor cantidad al tamaño deseado. Requiere de una búsqueda exhaustiva.
- Peor ajuste: se asigna el hueco cuyo tamaño exceda en la mayor cantidad al tamaño deseado. Requiere también de una búsqueda exhaustiva.
- El siguiente ajuste: es igual que el «primer ajuste» con la diferencia que se deja un apuntador al lugar en donde se asignó el último hueco para realizar la siguiente búsqueda a partir de él.
- Ajuste rápido: se mantienen listas enlazadas separadas de acuerdo a los tamaños de los huecos, para así buscarle a los procesos un hueco más rápido en la cola correspondiente.



Overlays

- ▶ Para ejecutar grandes cantidades de código en áreas de memoria muy pequeñas, auxiliados por algunas llamadas al sistema operativo.
- ▶ Es así como nacen los «overlays».

3. Administración de la Memoria: La Memoria Virtual

Fundamentos de Sistemas Operativos

Introducción

- ▶ Los esquemas para la asignación de memoria vistos en la lección anterior requerían que un programa que se fuera a ejecutar debía:
 - ▶ Ser cargado completamente en memoria primaria.
 - ▶ Estar en localizaciones continuas.
- ▶ Históricamente, el segundo de estas restricciones fue la primera en ser eliminada.

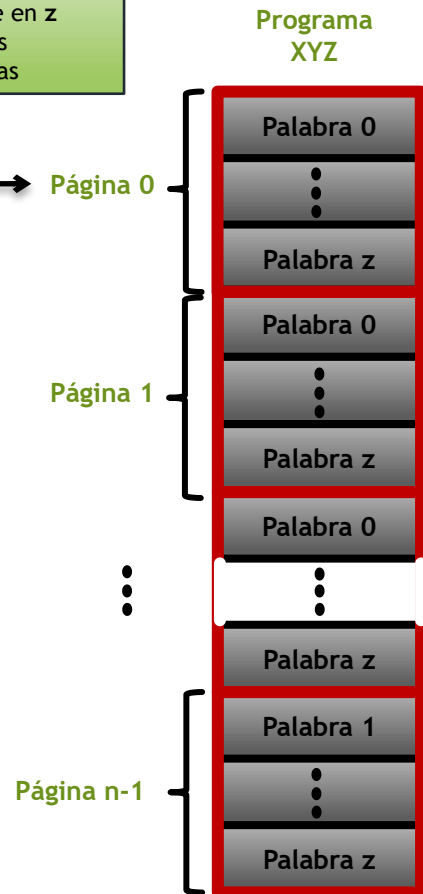
Asignación Paginada de Memoria

- ▶ En el esquema de **asignación paginada de memoria** (*paged memory allocation*) el programa que se carga a memoria se divide en porciones del mismo tamaño llamadas **páginas**.
 - ▶ Estas páginas **no** tienen que ser colocadas de forma continua en memoria.
 - ▶ Pero **todas** las páginas deben ser cargadas antes de ejecutar el programa.

Memoria virtual

Paginación

1. Cada página consiste en z palabras contiguas



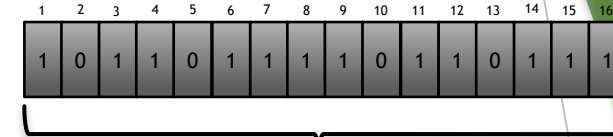
2. Un espacio de direcciones N de un programa consiste de n páginas ($0, 1, 2, 3, \dots, n-1$) ($n \cdot z$ direcciones virtuales)

Programa XYZ

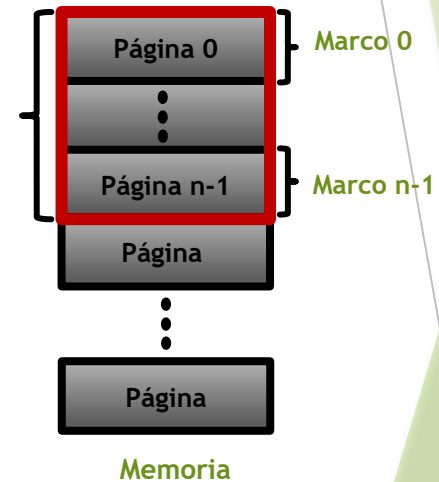
La técnica de paginación simple es similar a la partición estática.

Las diferencias están en que, con paginación las particiones son más pequeñas, un programa puede ocupar más de una partición y éstas no tienen por qué estar contiguas.

Los ordenadores modernos normalmente tienen un tamaño de palabra de 16, 32 ó 64 bits.



palabra



3. el espacio de memoria consiste de m marcos de páginas ($0, z, 2z, \dots, (m-1)z$) ($m \cdot z$ posiciones)

Asignación Paginada de Memoria (cont.)

- ▶ La mayoría de los s.o. escogen un tamaño de página que sea igual al tamaño de un **sector** (bloque) de disco.
- ▶ Esto se hace para que cada página sea cargada con una sola operación de lectura de disco.

Asignación Paginada de Memoria (cont.)

- ▶ La memoria, a su vez, se divide en porciones del mismo tamaño de los sectores de disco llamadas **marcos de página** (*page frames*).
- ▶ Por lo tanto,
 - ▶ Un sector de disco contendrá
 - ▶ Una página de instrucciones que cabrá en
 - ▶ Un marco de página de memoria primaria

Asignación Paginada de Memoria (cont.)

- ▶ Antes de ejecutar un programa el Administrador de la Memoria:
 1. Determina la cantidad de páginas en el programa.
 2. Localiza los marcos de página en la memoria primaria.
 3. Carga todas las páginas a sus respectivos marcos.

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

(a) 15 páginas libres

0	A.0
1	A.1
2	A.2
3	A.3
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

b) Carga del proceso A

0	A.0
1	A.1
2	A.2
3	A.3
4	B.0
5	B.1
6	B.2
7	
8	
9	
10	
11	
12	
13	
14	

c) Carga del proceso B

0	A.0
1	A.1
2	A.2
3	A.3
4	B.0
5	B.1
6	B.2
7	C.0
8	C.1
9	C.2
10	C.3
11	
12	
13	
14	

d) Carga del proceso C

0	A.0
1	A.1
2	A.2
3	A.3
4	
5	
6	
7	C.0
8	C.1
9	C.2
10	C.3
11	
12	
13	
14	

e) Expulsión del proceso B

0	A.0
1	A.1
2	A.2
3	A.3
4	D.0
5	D.1
6	D.2
7	C.0
8	C.1
9	C.2
10	C.3
11	D.3
12	D.4
13	
14	

f) Carga del proceso D

0	0	Tablas del páginas del Proceso A
1	1	
2	2	
3	3	

0	--	Tablas del páginas del Proceso B
1	--	
2	--	

0	7	Tablas del páginas del Proceso C
1	8	
2	9	
3	10	

0	4	Tablas del páginas del Proceso D
1	5	
2	6	
3	11	
4	12	

13
14

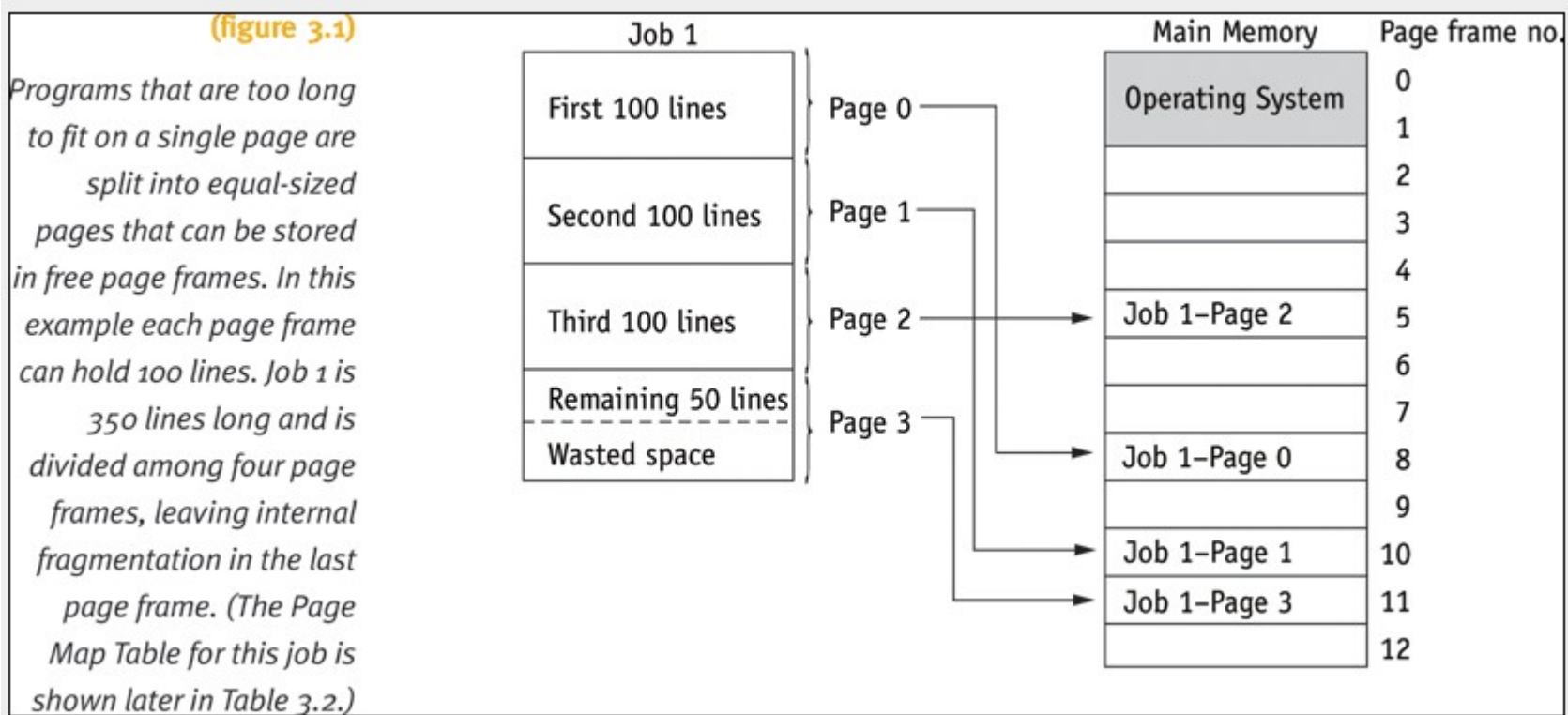
Listas de marcos
libres

Asignación Paginada de Memoria (cont.)

- ▶ Como los marcos de página pueden ser reutilizados tan pronto quedan disponibles no hay fragmentación externa.
- ▶ Como las páginas no tienen que estar continuas en memoria no hay necesidad de relocalizar.
- ▶ La dificultad en este esquema es cómo el Administrador de la Memoria monitorea dónde está cada página.

Asignación Paginada de Memoria (cont.)

- La siguiente figura muestra que un problema (pero pequeño) es la fragmentación interna del último marco de página.



Asignación Paginada de Memoria (cont.)

- ▶ Hay tres tablas que deben ser manejadas:
 - ▶ **Tabla de Trabajos** (*Job Table*, JT)
 - ▶ **Tabla de Correspondencia de Páginas** (*Page Map Table*, PMT)
 - ▶ **Tabla de Correspondencia de la Memoria** (*Memory Map Table*, MMT)

Asignación Paginada de Memoria (cont.)

- ▶ La Tabla de Trabajos (JT) contiene, para cada programa en memoria:
 - ▶ El número del programa (*job*)
 - ▶ Su tamaño
 - ▶ La localización de su Tabla de Correspondencia de Páginas (PMT)

Job Table			Job Table		Job Table		(table 3.1)
Job Number	Job Size	PMT Location	Job Size	PMT Location	Job Size	PMT Location	
0	400	3096	400	3096	400	3096	This section of the Job Table (a) initially has three entries, one for each job in process. When the second job ends (b), its entry in the table is released and it is replaced (c) by information about the next job that is to be processed.
1	200	3100			700	3100	
2	500	3150	500	3150	500	3150	
(a)			(b)		(c)		

Asignación Paginada de Memoria (cont.)

- ▶ Cada programa tiene su propia Tabla de Correspondencia de Páginas (PMT) que contiene:
 - ▶ El número de cada página
 - ▶ El número del marco en memoria primaria donde se cargó

(table 3.2)

Page Map Table for Job 1
in Figure 3.1.

Job Page Number	Page Frame Number
0	8
1	10
2	5
3	11

Asignación Paginada de Memoria (cont.)

- ▶ La Tabla de Correspondencia de la Memoria contiene (MMT):
 - ▶ El número de cada marco de página
 - ▶ El estatus (libre/ocupado) de cada marco de página

Page Frame Number	Status
0	Free
1	Free
2	Free
3	Free
4	Free
5	Busy
6	Free
7	Free
8	Busy
9	Free
10	Busy
11	Busy

Asignación Paginada de Memoria (cont.)

- ▶ Si se divide el número de una línea de código por el tamaño de la página,
 - ▶ El cociente es el número de la página
 - ▶ El residuo es el **desplazamiento** (*displacement, offset*) de la línea de código con respecto al comienzo de la página
- ▶ Como se ve en la próxima figura, las páginas y las líneas dentro de cada página se numeran comenzando en 0.

Asignación Paginada de Memoria (cont.)

- ▶ Para calcular la **localización absoluta** de la línea en memoria:
 - ▶ Hay que buscar en el PMT del programa cuál marco contiene la página.
 - ▶ Hay que obtener la dirección del marco de página:

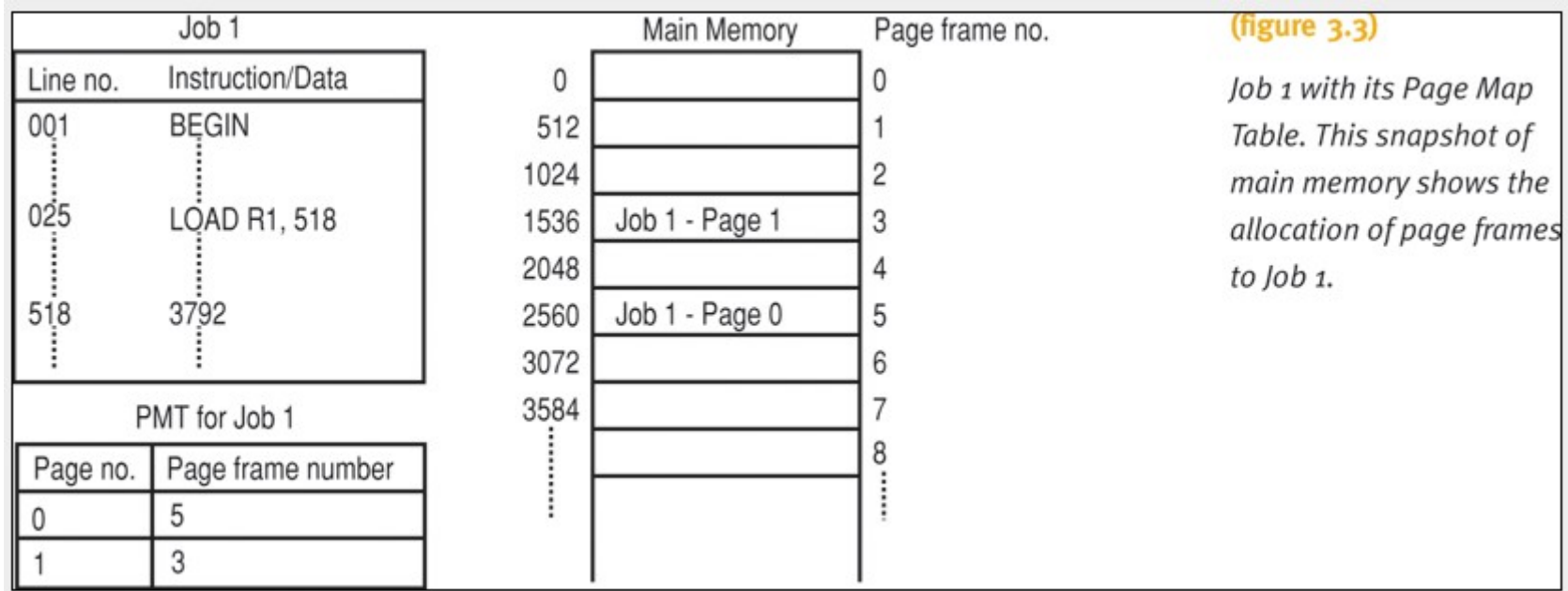
$\text{addressOfPageFrame} = \text{pageFrameNum} * \text{pageSize}$

- ▶ Hay que obtener la dirección de la línea dentro del marco:

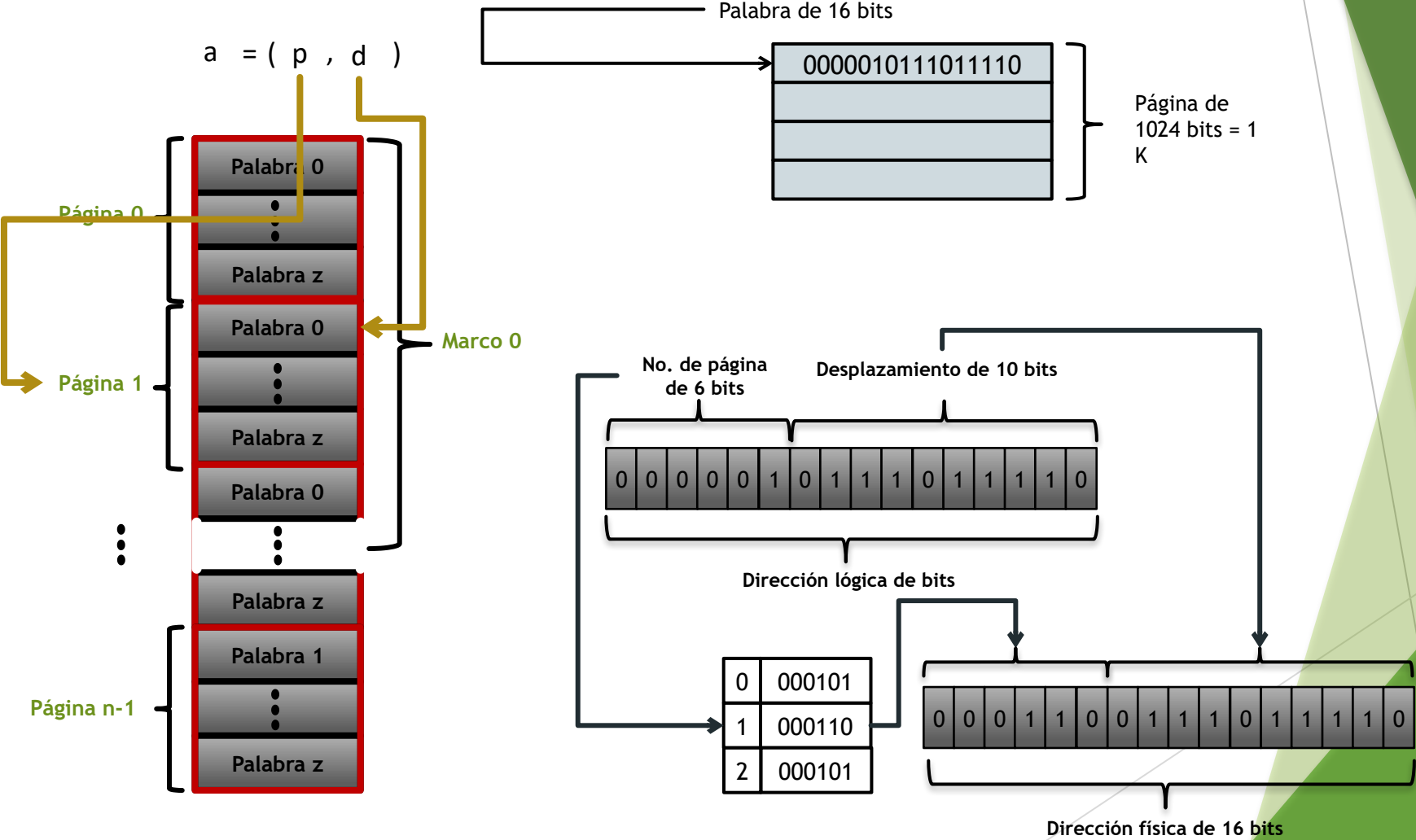
$\text{addressOfInstruction} = \text{addressOfPageFrame} + \text{displacement}$

Asignación Paginada de Memoria (cont.)

- ¿Cuál es la localización de la línea 518 en memoria, asumiendo que cada marco contiene 512 líneas? La dirección 1542 .



Una dirección virtual a es equivalente a una dirección dada como una dupla (p, d) , en la cual p es el número de la página y d el número de la palabra dentro de la página



Asignación Paginada de Memoria (cont.)

- ▶ Cada vez que se ejecuta una instrucción o se utiliza un valor de datos, el s.o. debe traducir las direcciones relativas que maneja el programa a direcciones absolutas.
- ▶ Esto se conoce como **resolución de direcciones** (*address resolution*).

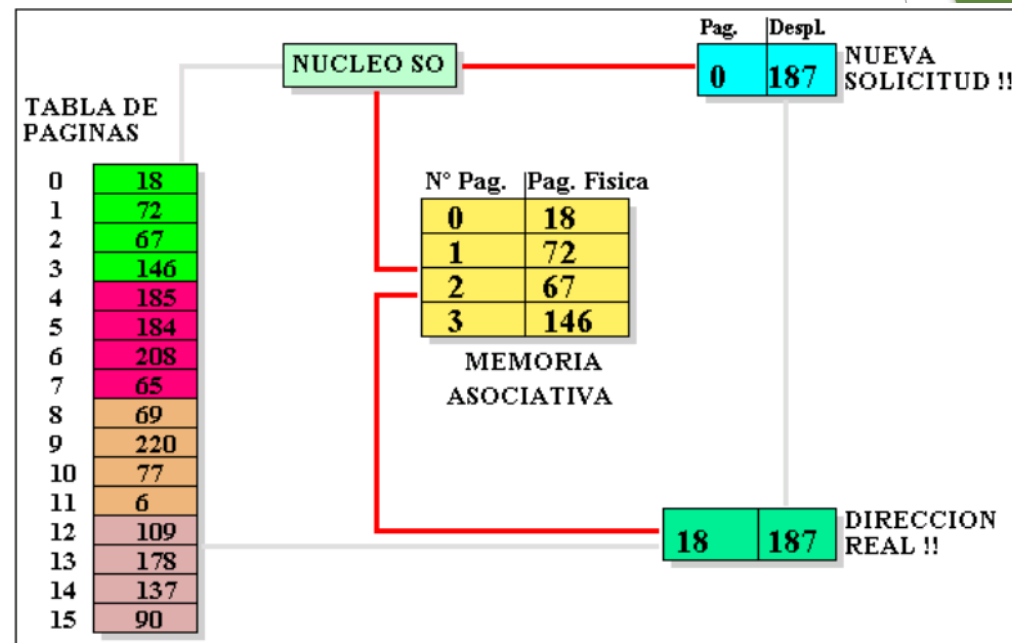
Asignación Paginada de Memoria (cont.)

- ▶ En muchos sistemas de computadoras se utilizan registros para la asignación paginada de memoria.
- ▶ Esto aligera la resolución de direcciones pero requiere que la arquitectura de la computadora asuma que se usará el esquema de paginación.

Memoria Asociativa

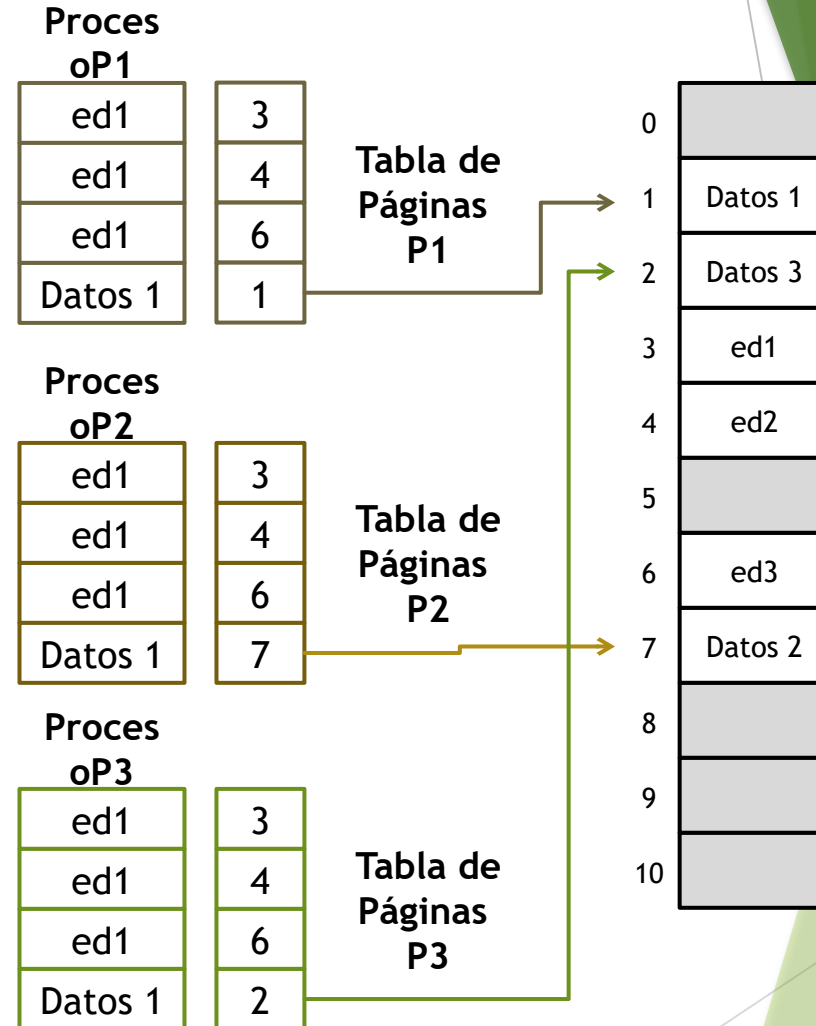
En la práctica es necesario que la mayoría de los accesos a memoria no impliquen una consulta a la tabla de páginas, sino que únicamente requieran el acceso a la posición solicitada. Esto se logra mediante el uso de una memoria asociativa interna en la MMU que mantiene información sobre las últimas páginas accedidas.

Esta memoria asociativa es considerada memoria cache especial de alta velocidad.



Páginas compartidas

Otra ventaja de la paginación es la posibilidad de compartir programas de uso común. Esta técnica resulta sumamente importante en un entorno de tiempo compartido. Se pueden compartir desde editores como compiladores, ensambladores y sistemas de base de datos. Para que sean compartibles, el código debe ser reentrante (no automodificable), es decir, el código es sólo de búsqueda o sólo lectura.



Paginación por Demanda

- ▶ La **paginación por demanda** (*demand paging*) permite cargar sólo una parte del programa en memoria.
- ▶ En este esquema el programa sigue dividiéndose en porciones de tamaño fijo (páginas) que inicialmente residen en memoria secundaria.
- ▶ Estas páginas sólo se cargan cuando sea necesario.

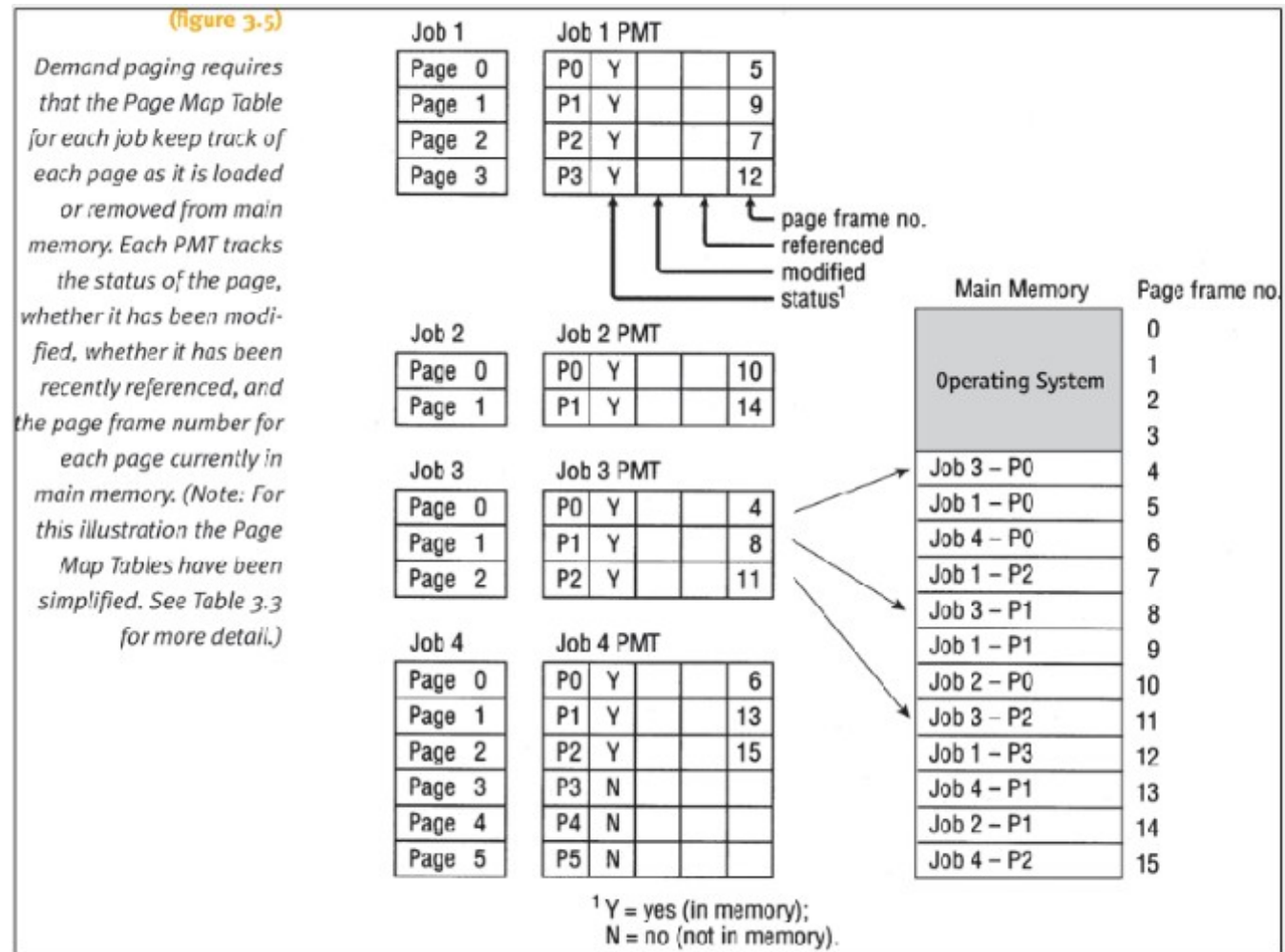
Paginación por Demanda (cont.)

- ▶ La clave de la paginación por demanda requiere un acceso de alta velocidad a las páginas que están en disco.
- ▶ Este esquema hizo viable la **memoria virtual** (*virtual memory*) ya que permite dar la apariencia de tener disponible una cantidad ilimitada de memoria al utilizar parte de la memoria secundaria como resguardo de la memoria primaria.

Paginación por Demanda (cont.)

- ▶ Como en el esquema tradicional de paginación, el s.o. mantiene las tablas JT, PMT y MMT.
- ▶ En el caso de la Tabla de Correspondencia de Páginas, cada entrada tiene tres columnas adicionales:
 - ▶ Bit para determinar si la página ya está en memoria (bit de estatus de presencia)
 - ▶ Bit para determinar si el contenido de la página ha sido modificado
 - ▶ Bit para determinar si la página ha sido accedida (referenciada) recientemente

Paginación por Demanda (cont.)



Paginación por Demanda (cont.)

- ▶ El **bit de estatus de página** (*in-memory status bit*) se utiliza para saber si hay que buscar en disco la página o si sólo hay que localizarla en memoria principal, lo que es mucho más rápido.
- ▶ El **bit de página modificada** (*modified bit*) es utilizado cuando hay que sacar una página de memoria. Si no ha sido modificada, no hay que reescribir la copia en disco.
- ▶ El **bit de página referenciada** (*referenced bit*) se utiliza para determinar si la página es o no candidata a remoción.

Paginación por Demanda (cont.)

- ▶ Cada vez que se necesita acceder a una instrucción o a un dato hay que buscarlo en memoria.
- ▶ Cuando el s.o. no lo encuentra, ocurre lo que se conoce como **falla de página** (*page fault*).
- ▶ Si hay marcos disponibles, el s.o. simplemente busca la página y la carga al marco correspondiente.

Paginación por Demanda (cont.)

- ▶ Si no hay marcos disponibles, el s.o. determina cuál marco debe vaciarse para hacer espacio para la página.
- ▶ En caso de que el marco contenga una página que ha sido modificada, se genera una interrupción y hay que pasar la copia modificada a disco antes de cargar la nueva página de disco a memoria.
- ▶ Este proceso se conoce como **intercambio** (*swapping*).

Paginación por Demanda (cont.)

- ▶ El proceso de intercambio requiere que se actualice el PMT para ambos programas (el que tiene la página que se remueve y el que tiene la página que se carga).
- ▶ Si hay una cantidad excesiva de intercambios de página entre la memoria principal y el almacenamiento secundario, ocurre lo que se conoce como *thrashing*. (Distorsion de swapping)

Paginación por Demanda (cont.)

- ▶ El problema del thrashing es que requiere mucho esfuerzo por parte de la computadora pero logra muy poco.
- ▶ La razón principal de los azotes es tener una gran cantidad de programas compitiendo por una pequeña cantidad de marcos en memoria.

Paginación por Demanda (cont.)

- El siguiente es el algoritmo que utiliza el s.o para procesar una instrucción:

```
For each instruction in job:  
  Compute page number  
  
  If page is in memory:  
    Fetch instruction from memory  
    Decode instruction  
    Execute instruction  
  Else  
    Generate page interrupt  
    Call page_fault_handler  
End For
```

Paginación por Demanda (cont.)

```
Procedure page_fault_handler:  
  If there is no free page frame:  
    Select page to be swapped out  
    Update job's Page Map Table  
  
    If content of page has been changed:  
      Write page to disk  
    End If  
  End If  
  
  Compute disk address for the page  
  Read page from disk into memory  
  Update job's Page Map Table  
  Update Memory Map Table  
  Restart interrupted instruction  
End Procedure
```

Políticas de Reemplazo de Páginas

- ▶ Como hemos visto, cuando todos los marcos están ocupados y se tiene que cargar una página a memoria, se debe seleccionar cuál de las páginas que ya están cargadas debe reemplazarse.
- ▶ Esta decisión es crucial para la eficiencia del sistema.

Políticas de Reemplazo de Páginas (cont.)

- ▶ Existen varios algoritmos para remoción de páginas, tales como:
 - ▶ Reemplazo óptimo (*optimal replacement*)
 - ▶ La primera página que llegó es la primera que sale (*first-in first-out*, FIFO)
 - ▶ Página menos usada recientemente (*least recently used*, LRU)

Políticas de Reemplazo de Páginas (cont.)

- ▶ La política del **reemplazo óptimo** (*optimal replacement*) selecciona para reemplazo aquella página que tardará más tiempo en ser referenciada en el futuro.
- ▶ Esta política es la mejor teóricamente pero es imposible de implementar ya que requiere que el s.o. sea clarividente.
- ▶ El interés de este algoritmo es que sirve para comparar el rendimiento de otros algoritmos.

Políticas de Reemplazo de Páginas (cont.)

- ▶ La política de que **la primera página que llegó es la primera que sale** (*first-in first-out*, FIFO), el s.o. mantiene una cola con un ID para cada página que se carga.
- ▶ En esta cola las páginas siempre se añaden al final, por lo que la página que más tiempo lleve cargada estará al inicio de la cola.
- ▶ Cuando hay que remover una página, el s.o. simplemente selecciona la primera página en la cola.

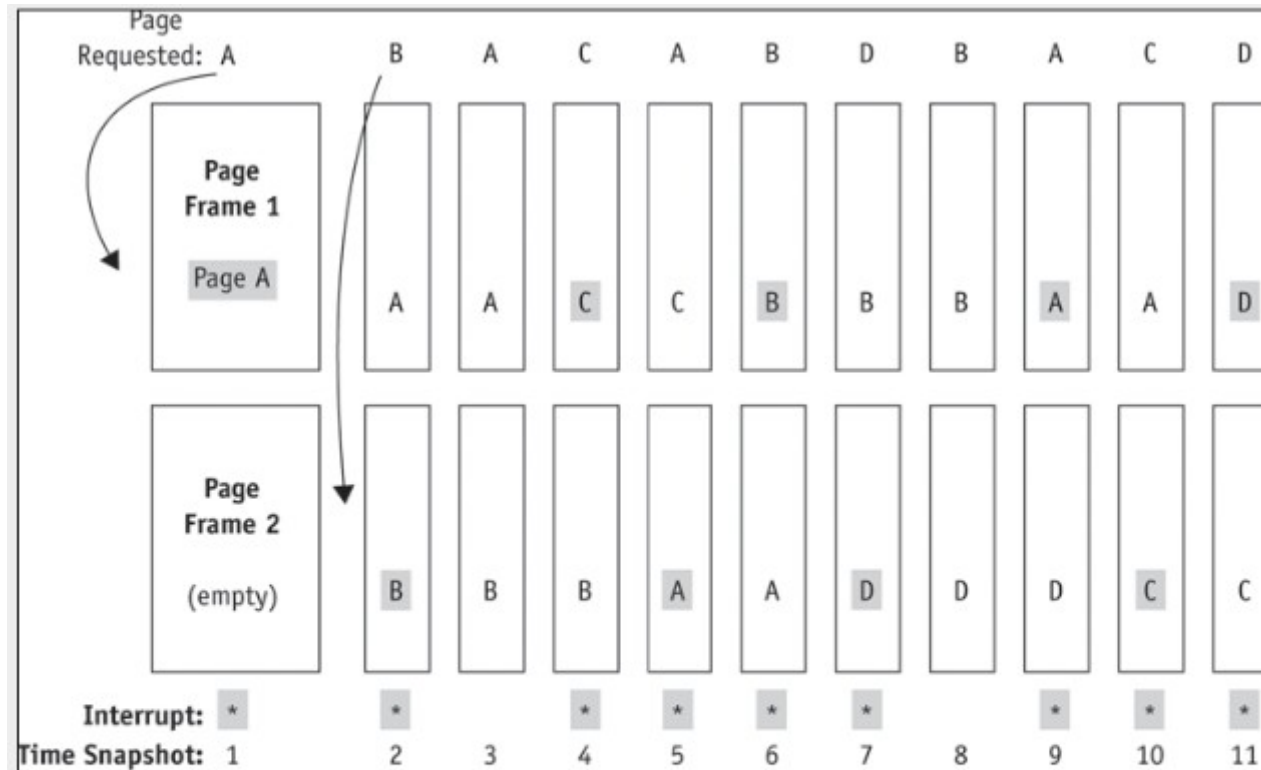
Políticas de Reemplazo de Páginas (cont.)

- ▶ Este algoritmo no toma en consideración cuántas veces ha sido referenciada la página que se selecciona para remoción.
- ▶ Además, la **anomalía de Belady** demuestra que hay situaciones en las cuales aumentar la cantidad de marcos en memoria sólo logra aumentar la cantidad de fallas (*page faults*).

Políticas de Reemplazo de Páginas (cont.)

- ▶ La siguiente figura muestra cómo muestra el algoritmo FIFO con un programa que tiene cuatro páginas (A, B, C y D) y que se desea cargar a una memoria con sólo dos marcos disponibles.
- ▶ El programa necesita procesar sus páginas en este orden: A, B, A, C, A, B, D, B, A, C, D.

Políticas de Reemplazo de Páginas (cont.)



(figure 3.8)

Using a FIFO policy, each page requested is swapped into the two available page frames. When the program is ready to be processed all four pages are in secondary storage. When the program calls a page that isn't already in memory, a page interrupt is issued, as shown by the gray boxes and asterisks. This program resulted in nine page interrupts.

Políticas de Reemplazo de Páginas (cont.)

- ▶ En la figura anterior, cada asterísco representa una falla de página.
- ▶ Por lo tanto, hay 9 fallas para 11 solicitudes de página (un 82% de **tasa de fallas** [*failure rate*]) lo cual es inaceptable.

Políticas de Reemplazo de Páginas (cont.)

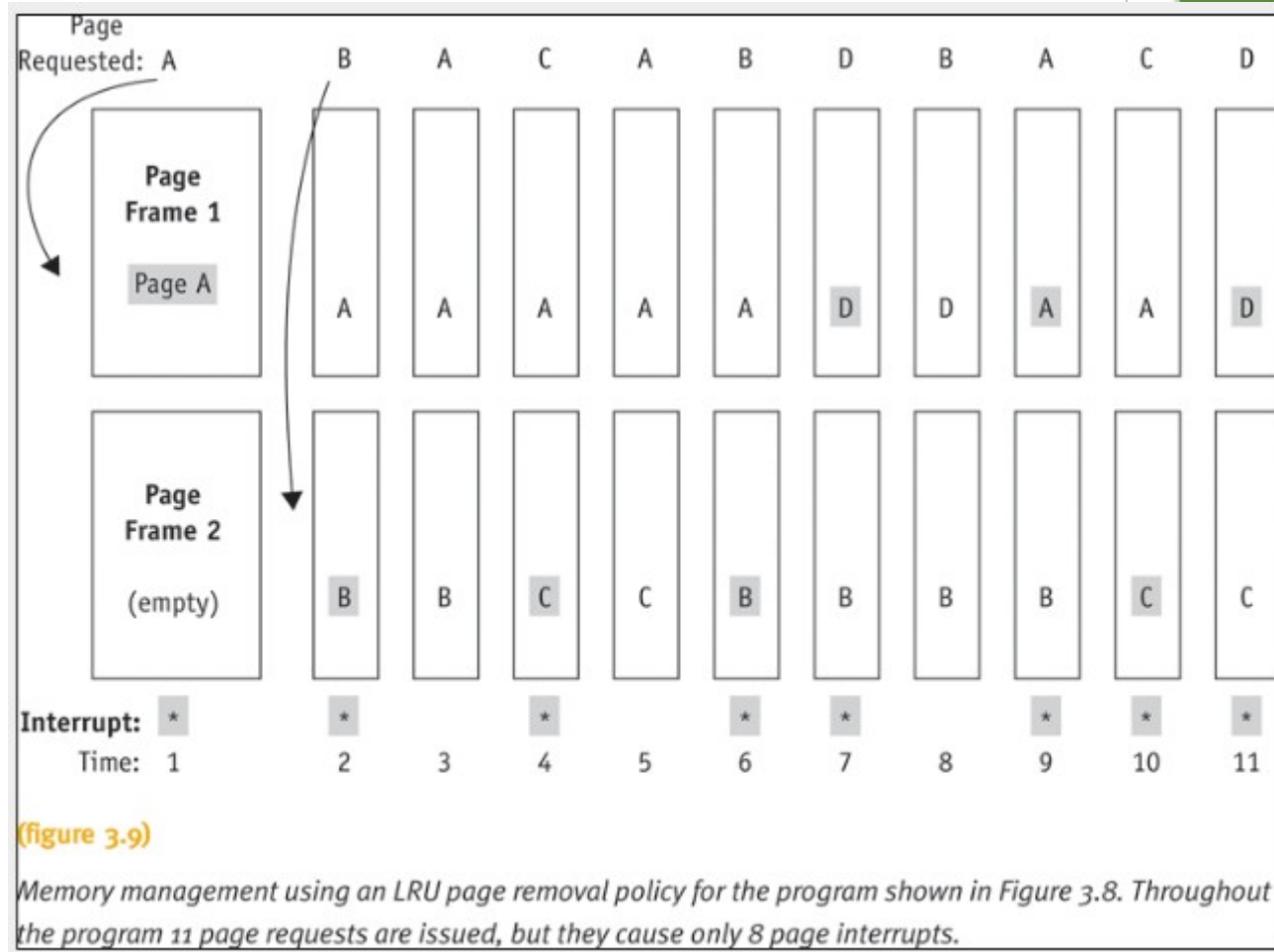
- ▶ La política de remoción de la **página menos usada recientemente** (*least recently used*, LRU, NRU Not Recently Use, Algoritmo Reloj) parte de la premisa que aquellas páginas que han sido usadas frecuentemente por las últimas instrucciones ejecutadas serán también usadas frecuentemente por las próximas instrucciones que se ejecutarán.
- ▶ Esto se conoce como el **principio de localidad** (*principle of locality*).

Políticas de Reemplazo de Páginas (cont.)

- ▶ Por lo tanto, hace sentido remover aquellas páginas que no han sido utilizadas recientemente pues se asume que no serán referenciadas en el futuro inmediato.
- ▶ El s.o. debe llevar un temporizador (*timer*) que indique cuándo fue la última vez que se referenció cada página para poder decidir cuál remover.

Políticas de Reemplazo de Páginas (cont.)

La siguiente figura muestra cómo el algoritmo LRU trabaja:



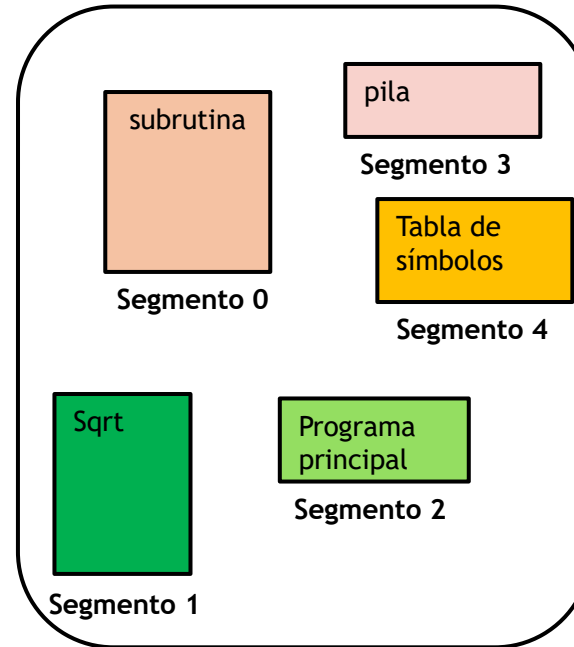
Asignación Segmentada de Memoria

- ▶ En el esquema de **asignación segmentada de memoria** (*segmented memory allocation*), cada programa se divide en varios segmentos de distinto tamaño.
- ▶ Muchas veces el criterio es segmentar el programa por módulos o subrutinas.
- ▶ Esto reduce las fallas de página que resultan de tener código relacionado que distribuido en más de una página.

Memoria virtual

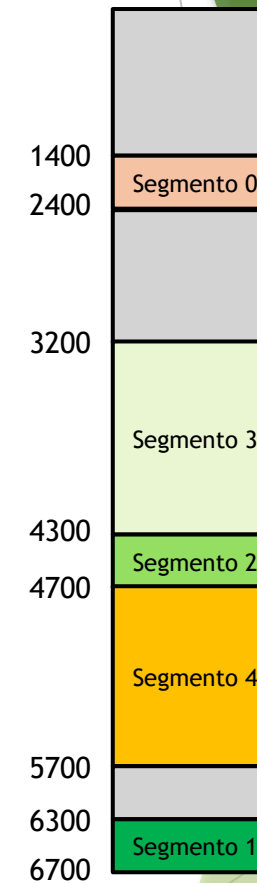
Segmentación

La **segmentación** es un esquema de administración de la memoria que soporta la visión que el usuario tiene de la misma. Un espacio de direcciones lógicas es una colección de segmentos. Cada segmento tiene un nombre y una longitud. Las direcciones especifican tanto el nombre del segmento como el desplazamiento dentro del segmento. Por lo tanto, el usuario especifica cada dirección mediante dos cantidades: un nombre de segmento y un desplazamiento.



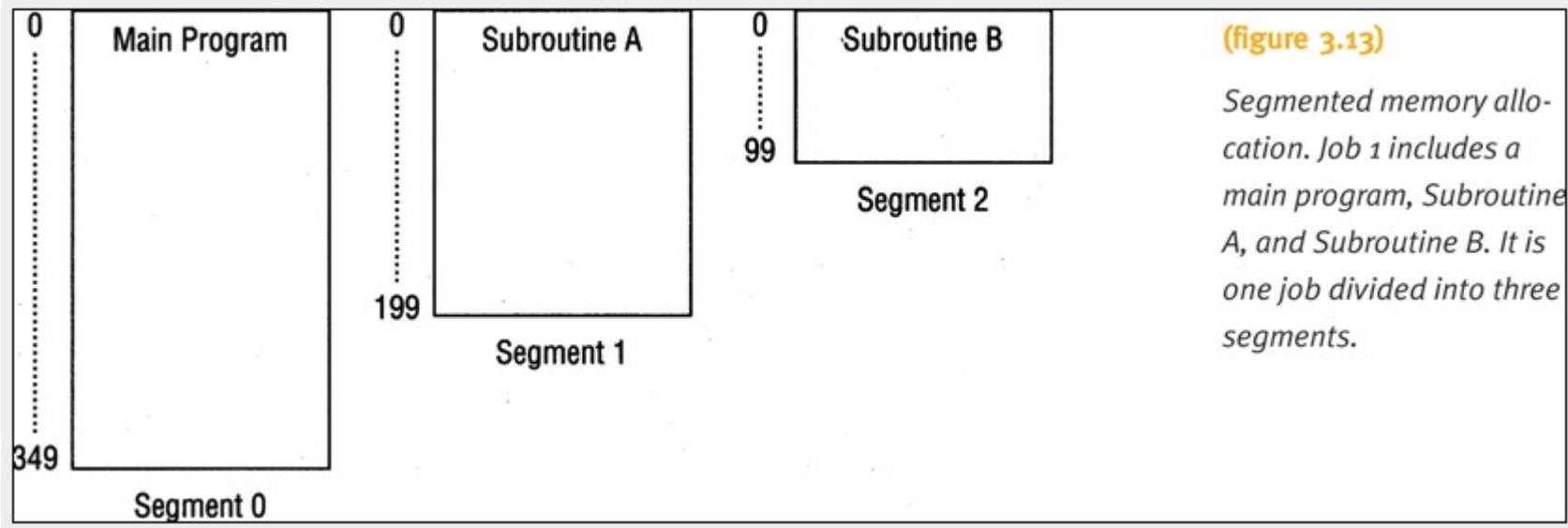
	Límite	Base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

Tabla de segmentos



Asignación Segmentada de Memoria

- Aquí se muestra un programa dividido en segmentos:



Asignación Segmentada de Memoria (cont.)

- ▶ Cuando se usa la segmentación, la memoria principal ya no se divide en marcos de página sino que se asigna dinámicamente.
- ▶ En este esquema, como en paginación, el s.o. mantiene tres tablas:
 - ▶ Tabla de Trabajos (*Job Table* o JT)
 - ▶ Tabla de Correspondencia de Segmentos (*Segment Map Table* o SMT)
 - ▶ Tabla de Correspondencia de Memoria (*Memory Map Table* o MMT)

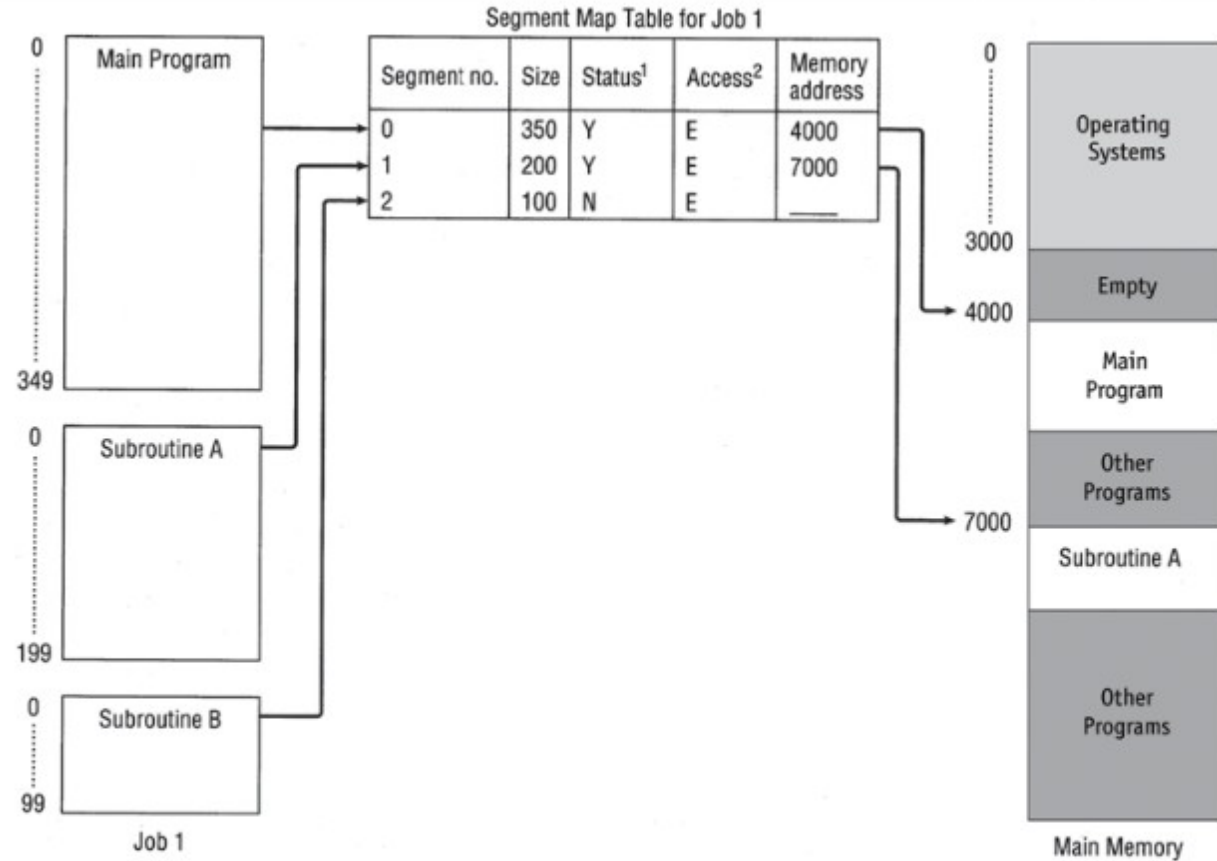
Asignación Segmentada de Memoria (cont.)

- ▶ La **tabla de correspondencia de segmentos** (*Segment Map Table* o SMT) es creada para cada programa.
- ▶ El SMT contiene:
 - ▶ El número de cada segmento
 - ▶ Su tamaño
 - ▶ Bits de estatus (presencia en memoria), referencia y modificación
 - ▶ Derechos de acceso (lectura, escritura, ejecución)
 - ▶ Localización en memoria

Asignación Segmentada de Memoria (cont.)

(figure 3.14)

The Segment Map Table tracks each segment for Job 1.



¹ Y = in memory;
N = not in memory.

² E = Execute only.

Asignación Segmentada de Memoria (cont.)

- ▶ El esquema de segmentación permite que los segmentos no estén colocados de forma continua en memoria principal y que sólo se carguen a medida de que se necesitan (de acuerdo a la demanda).
- ▶ Para encontrar la localización real en memoria de una línea de código se lleva a cabo un proceso similar al que se llevaba a cabo en la paginación.

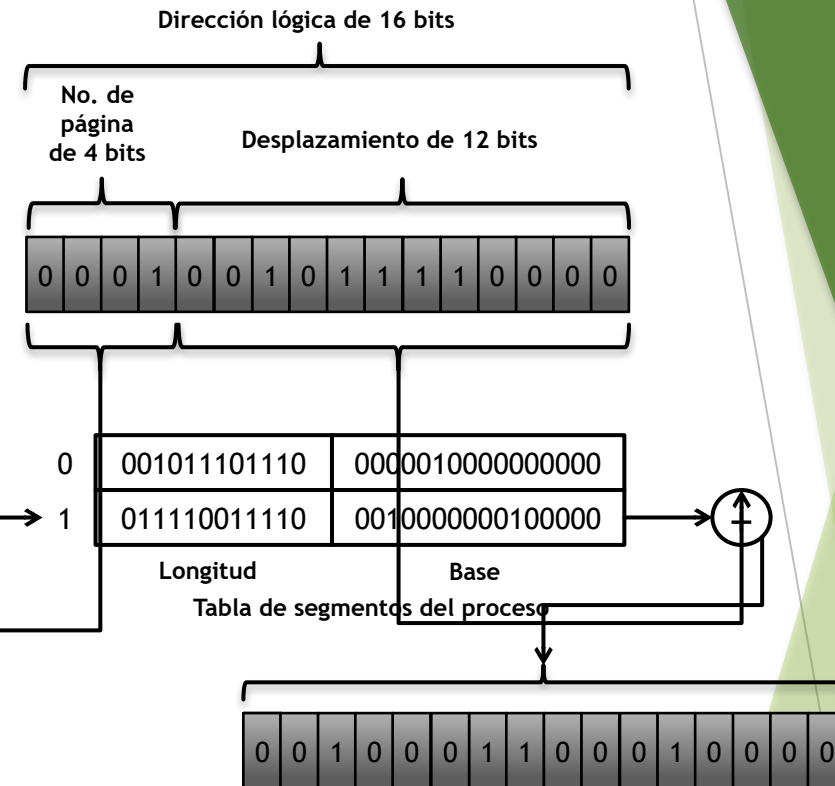
Asignación Segmentada de Memoria (cont.)

- ▶ Para calcular la **localización absoluta** de la línea en memoria:
 - ▶ Hay que buscar en el SMT del programa dónde comienza el segmento que contiene la línea.
 - ▶ Hay que obtener la dirección de la línea dentro del segmento de acuerdo a su desplazamiento:

$$\text{addressOfInstruction} = \text{beginLocationOfSegment} + \text{displacement}$$

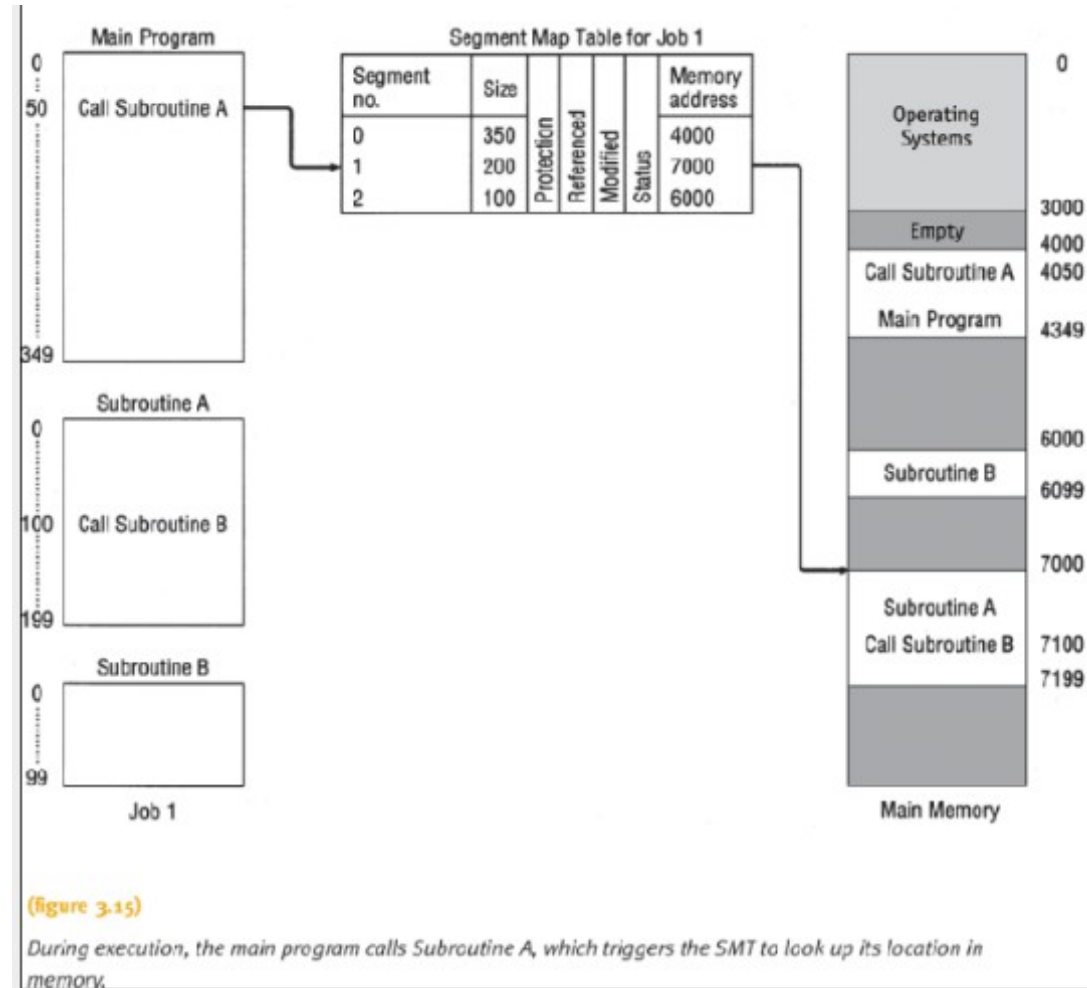
En forma análoga a la paginación, el proceso de traducción de direcciones en la segmentación, consiste en los siguientes pasos:

1. Extraer el número de segmentos de los n bits más significativos de la dirección lógica.
2. Emplear el número de segmentos como índice en la tabla de segmentos del proceso para obtener la dirección física de comienzo del segmento.
3. Comparar el desplazamiento expresado en los m bits menos significativos de la dirección lógica con la longitud del segmento, si es mayor, la dirección no es válida.
4. La dirección física buscada es la suma de la dirección física del inicio del segmento más el desplazamiento.



Cuando un proceso se introduce en memoria, se cargan todos sus segmentos en regiones de memoria libres y se crea la tabla de segmentos.

Asignación Segmentada de Memoria (cont.)



Asignación Segmentada de Memoria (cont.)

- ▶ Un problema que trae la asignación segmentada es que, como la memoria se divide dinámicamente, se forma fragmentación externa y es necesario recompactar la memoria cada cierto tiempo.
- ▶ Además normalmente se necesitan varias operaciones de acceso a memoria secundaria para cargar cada segmento.

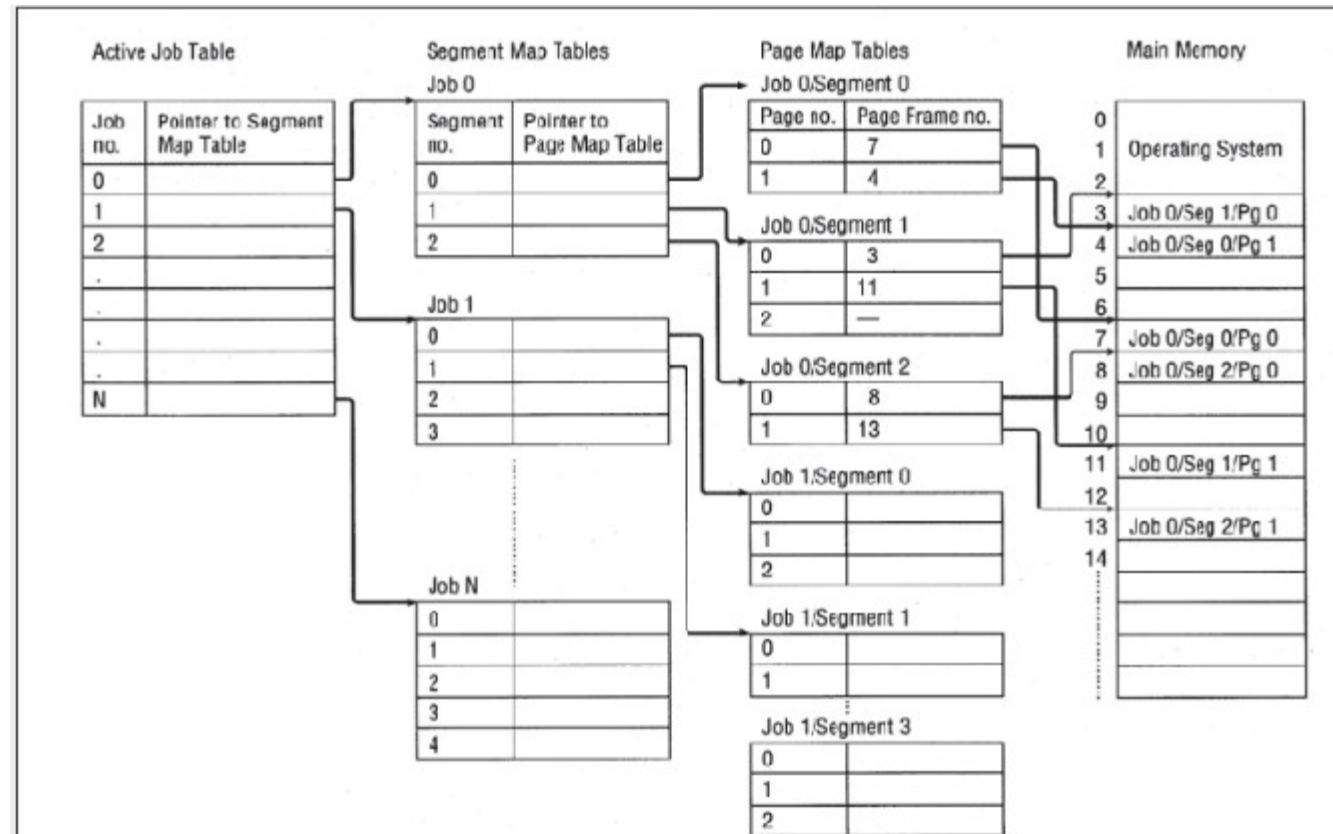
Asignación Segmentada y Paginada por Demanda de Memoria

- ▶ El esquema de **asignación segmentada y paginada por demanda** (*segmented/demand paged memory allocation*) divide lógicamente el programa en segmentos de distinto tamaño que forman grupos relacionados de instrucciones (subrutinas).
- ▶ Pero estos segmentos son divididos físicamente en páginas de tamaño fijo para su manipulación eficiente.
- ▶ No todas las páginas de un segmento tienen que estar en memoria a la vez.

Asignación Segmentada y Paginada por Demanda de Memoria (cont.)

- ▶ Este esquema resuelve muchos de los problemas de la segmentación en términos de manejo de memoria.
- ▶ Sin embargo, ahora el s.o. debe administrar cuatro tablas:
 - ▶ *Job Table* – una para el sistema, lista cada programa en memoria
 - ▶ *Segment Map Table* – una para cada programa, lista los detalles de cada segmento
 - ▶ *Page Map Table* – una para cada segmento, lista los detalles de cada página
 - ▶ *Memory Map Table* – una para el sistema, indica cómo se asignaron los marcos de página

Asignación Segmentada y Paginada por Demanda de Memoria (cont.)



(figure 3.16)

How the Job Table, Segment Map Table, Page Map Table, and main memory interact in a segment/paging scheme.

Asignación Segmentada y Paginada por Demanda de Memoria (cont.)

- ▶ Para determinar la localización en memoria de una instrucción o un dato, el s.o. debe:
 - ▶ Determinar el segmento en que se localiza.
 - ▶ Ir a la página adecuada dentro del segmento.
 - ▶ Moverse dentro de la página usando el desplazamiento adecuado.

Asignación Segmentada y Paginada por Demanda de Memoria (cont.)

- ▶ La mayor desventaja de este esquema es el esfuerzo extra (*overhead*) que se requiere para la manipulación de las tablas y el tiempo que requiere buscar una localización de memoria.
- ▶ Muchos sistemas usan memoria asociativa para acelerar este proceso.

Asignación Segmentada y Paginada por Demanda de Memoria (cont.)

- ▶ La **memoria asociativa o direccionada por contenido** (*associative memory, content-addressable memory*) es un tipo de memoria en la cual los datos no se identifican por una dirección sino por una etiqueta.
- ▶ Este tipo de memoria permite una búsqueda rápida de información.
- ▶ Los s.o. almacenan la información de las últimas páginas referenciadas en esta memoria para su rápido acceso.

Memoria Virtual

- ▶ Los esquemas de paginación y segmentación por demanda hacen posible tratar una porción de la memoria secundaria como si fuera parte de la memoria principal.
- ▶ A esta porción se le conoce como **memoria virtual**.
- ▶ El uso de la memoria virtual requiere la cooperación constante del Administrador de la Memoria (que determina dónde está cada página o segmento) y el CPU (que genera interrupciones en caso de intentos fallidos en el acceso de páginas y ayuda en la resolución de direcciones).

Memoria Virtual (cont.)

- La siguiente tabla muestra una comparación entre el uso de paginación (por demanda) y segmentación para implementar memoria virtual:

(table 3.6) <i>Comparison of virtual memory with paging and segmentation.</i>	Virtual Memory with Paging	Virtual Memory with Segmentation
	Allows internal fragmentation within page frames	Doesn't allow internal fragmentation
	Doesn't allow external fragmentation	Allows external fragmentation
	Programs are divided into equal-sized pages	Programs are divided into unequal-sized segments that contain logical groupings of code
	Absolute address calculated using page number and displacement	Absolute address calculated using segment number and displacement
	Requires PMT	Requires SMT

Memoria Virtual (cont.)

- ▶ Ventajas de la memoria virtual:
 - ▶ El tamaño del programa no está limitado por el tamaño de RAM
 - ▶ La memoria se usa eficientemente (acceso por demanda)
 - ▶ Elimina la fragmentación externa y minimiza la fragmentación interna (especialmente si se usa segmentación y paginación combinada)
 - ▶ Permite la multiprogramación (permite que varios programas o varios usuarios compartan la atención del CPU)

Memoria Virtual (cont.)

- ▶ Ventajas de la memoria virtual (cont.):
 - ▶ Permite compartir código de un programa entre varios usuarios (**código reentrante** [*reentrant code*], se mantiene una sola copia del código ejecutable pero varias copias de las áreas de datos para los usuarios).
 - ▶ Permite compartir segmentos entre varios programas y cargarlos por demanda (**enlace dinámico**, *dynamic linking*).

Memoria Virtual (cont.)

- ▶ Desventajas de la memoria virtual:
 - ▶ Aumento en la complejidad (y costo) del CPU
 - ▶ Esfuerzo extra para atender las interrupciones
 - ▶ Aumento en la complejidad del sistema operativo
- ▶ Estas desventajas son sobrepasadas por mucho por las ventajas.

Comparación de los Esquemas de Asignación de Memoria

(table 3.8)

Comparison of memory allocation schemes discussed in Chapters 2 and 3.

Scheme	Problem Solved	Problem Created	Changes in Software
Single-user contiguous		Job size limited to physical memory size; CPU often idle	None
Fixed partitions	Idle CPU time	Internal fragmentation; Job size limited to partition size	Add Processor Scheduler; Add protection handler
Dynamic partitions	Internal fragmentation	External fragmentation	None
Relocatable dynamic partitions	Internal fragmentation	Compaction overhead; Job size limited to physical memory size	Compaction algorithm
Paged	Need for compaction	Memory needed for tables; Job size limited to physical memory size; Internal fragmentation returns	Algorithms to handle Page Map Tables

Comparación de los Esquemas de Asignación de Memoria (cont.)

Demand paged	Job size no longer limited to memory size; More efficient memory use; Allows large-scale multiprogramming and time-sharing	Larger number of tables; Possibility of thrashing; Overhead required by page interrupts; Necessary paging hardware	Page replacement algorithm; Search algorithm for pages in secondary storage
Segmented	Internal fragmentation	Difficulty managing variable-length segments in secondary storage; External fragmentation	Dynamic linking package; Two-dimensional addressing scheme
Segmented/ demand paged	Large virtual memory; Segment loaded on demand	Table handling overhead; Memory needed for page and segment tables	Three-dimensional addressing scheme

Conclusión

- ▶ El Administrador de la Memoria (*Memory Manager*) tiene la tarea de asignar la memoria a cada programa que debe ser ejecutado y reclamarla cuando termina su ejecución.
- ▶ Una vez que los programas han sido cargados a memoria usando un esquema de asignación, otro módulo del s.o. debe asignar el CPU a cada programa de la forma más eficiente posible.